Cheatsheet · 33 chapters on one page

Natural Language Processing cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

163 key ideas | 232 formulas | 112 tables | 90 tips and traps

STEP 1 NLU · Language and meaning

1.1 What language is

The working definition, six levels from phonetics to pragmatics, symbols and the semiotic triangle, the speaker and hearer chain, Zipf and Grice, and recursion.

KEY IDEAS

- **Working definition** Language is a systematic, symbolic and structured system that starts as thought, organises it, and lets it be rebuilt in another mind.
- **Six levels** Phonetics, phonology, morphology, syntax, semantics, pragmatics. Each combines the units of the level below.
- **Two functions** Symbolic: words stand for concepts, not directly for things (semiotic triangle). Interactive: a speaker encodes and a hearer reconstructs, never an exact copy.
- **Form and meaning** A symbol is a conventional form meaning pair. Unicorn has a sense without a reference. TAM marks present, past and imagined reality.
- **Effort and clarity** Zipf: least effort, f(r) ≈ C ÷ r. Grice: quantity, quality, relation, manner.
- **Recursion** A clause inside a clause without limit. Everett claimed Pirahã lacks it, a challenge to Hauser, Chomsky and Fitch. Still debated.

FORMULAS

Working definition from the lecture

Exam favourite

Language is a **systematic, symbolic and structured** system that (1) starts as thought in the mind, (2) organises chaotic ideas into structure, and (3) lets those ideas be sent out, carried and rebuilt in another mind.

Every later topic in this course, from tokens to parse trees, is a way of recovering some part of this structure from text.

Lesson: What language is

The key claim Exam favourite

Language does not directly represent the external world. It represents our **conceptualised experience** of the world.

Say "leopard" in a classroom. No leopard appears, yet everyone understands, because the word calls up a concept that speaker and listener already share.

Lesson: The symbolic function: words stand for concepts

The communication chain Exam favourite

Thought → linguistic representation → signal → linguistic representation → reconstructed thought

The first and last boxes are different. Communication never copies a thought. It only gives the hearer enough to rebuild an approximation.

Lesson: The interactive function: from one mind to another

Zipf: least effort

$$f(r) \approx \frac{C}{r^{\alpha}}, \alpha \approx 1$$

A few short words such as "the" are used constantly. Thousands of rare words are kept for precision. Chapter 3.1 measures this on real text.

Lesson: The interactive function: from one mind to another

Grice: the cooperative principle

Quantity, Quality, Relation, Manner

Say enough but not too much, be truthful, be relevant, be clear. Breaking a maxim on purpose creates an implied meaning.

Lesson: The interactive function: from one mind to another

TABLES

Table: question · what it needs · where it leads

Question	What it needs	Where it leads
How do I manipulate this string of words?	String operations, patterns, counts and probabilities	Statistical NLP: n-gram models, classifiers and today the next word prediction of large language models
What kind of system could produce this behaviour?	A theory of the representations and rules inside a mind or machine	Symbolic NLP, formal grammars and the cognitive science side of linguistics

Lesson: What language is

Table: level · unit studied · question it answers · example

Level	Unit studied	Question it answers	Example
Phonetics	Raw speech sounds	How are sounds produced, carried and heard?	[k] is made by stopping air at the back of the mouth with no voice
Phonology	Phonemes, the sound system	Which sound differences change meaning in this language?	/p/ against /b/ turns "pat" into "bat"
Morphology	Words and word parts	How are words built from meaningful parts?	dis + taste = distaste
Syntax	Phrases and sentences	How do words combine into sentences?	"The cat jumped" is subject then verb
Semantics	Literal meaning	What does the sentence mean on its own?	"The cat jumped" names a jumping event done by a cat
Pragmatics	Meaning in context	What does the speaker intend here and now?	"I sat near a bank": river or money?

Lesson: The six levels of linguistic structure

Table: reality · example from the lecture · how grammar marks it

Reality	Example from the lecture	How grammar marks it
Present	The cat is sleeping.	Present tense with progressive aspect (is + verb + ing)
Past	The cat was sleeping.	Past tense with progressive aspect (was + verb + ing)
Imagined	Imagine a cat flying over the moon.	Irrealis mood, signalled by "imagine"
Abstract	Justice is important.	A property turned into a noun with no physical referent

Lesson: The symbolic function: words stand for concepts

TIPS AND TRAPS

REMEMBER

Structure creates meaning. "Dog bites man" and "man bites dog" use the same three words. A bag of words model that ignores order cannot tell them apart, which is why later chapters bother with tags, n-grams and parse trees.

Lesson: What language is

COMMON MISTAKE Listeners usually also infer "if it does not rain, I will go", which the logic does not say. This gap between logical form and everyday meaning is one reason formal logic alone could not solve language understanding.

Lesson: The symbolic function: words stand for concepts

IN THE EXAM In an essay, link Zipf to the speaker side (minimise effort) and Grice to the shared contract between both sides.

Lesson: The interactive function: from one mind to another

1.2 Ambiguity: why language is hard for machines

Lexical against structural ambiguity, the Lesk method, PP attachment, Catalan growth of parse trees and how machines resolve ambiguity.

KEY IDEAS

- **Two kinds** Lexical: one word, several meanings (bank, over). Structural: one string, several trees (telescope).
- **Word types** Homonymy (unrelated meanings), polysemy (related senses), homophones (same sound), homographs (same spelling).
- **Lesk** Pick the sense whose gloss shares most words with the context. A counting version of Firth.
- **Catalan growth** A verb, object and k prepositional phrases have C(k + 1) trees: 2, 5, 14, 42, 132.
- **Resolution** Rules (right association), treebank statistics (lexical association), contextual neural models. Ask when the cost of a wrong guess is high.
- **Lecture examples** I sat near a bank. The cat jumped over the wall. John saw Mary with a telescope. Understands you like your mother. Copy the local patient files.

FORMULAS

Simplified Lesk Exam favourite

$$\text{sense}^* = \operatorname{argmax}_s |\text{context}(w) \cap \text{gloss}(s)|$$

Count the shared words between the sentence and each gloss, after removing very common words. Ties and zero overlaps fall back to the most frequent sense.

Lesson: Lexical ambiguity: one word, several meanings

Catalan numbers Exam favourite

$$C_n = \frac{1}{n+1} \times (2n \text{ choose } n) = \frac{(2n)!}{(n+1)!\, n!}$$

With k phrases the count is C_{k+1}: 2, 5, 14, 42, 132 for $k = 1$ to 5.

Lesson: Structural ambiguity: one string, several trees

Lexical association for attachment

$$\text{P(verb attachment)} \approx \frac{c(\text{verb, prep})}{c(\text{verb, prep}) + c(\text{noun, prep})}$$

In the spirit of Hindle and Rooth (1993): compare how strongly the preposition goes with the verb and with the noun in a large corpus.

Lesson: How machines resolve ambiguity

TABLES

Table: term · what it means · example

Term	What it means	Example
Homonymy	Same spelling and sound, unrelated meanings with different histories	bank (river, from Old Norse banki) and bank (money, from Italian banca, a bench)
Polysemy	One word, several related senses from one origin	mouth of a person and mouth of a river, head of a company
Homophone	Same sound, different spelling and meaning	pear and peer, write and right
Homograph	Same spelling, different sound and meaning	lead the metal and lead the verb

Lesson: Lexical ambiguity: one word, several meanings

Table: reading · where the phrase attaches · meaning

Reading	Where the phrase attaches	Meaning
Instrument	To the verb phrase (high attachment)	John used a telescope to see Mary
Possession	To the noun phrase Mary (low attachment)	John saw Mary, who had a telescope

Lesson: Structural ambiguity: one string, several trees

Table: era · approach · how it decides

Era	Approach	How it decides
1960s to 1980s	Rules and heuristics	Hand written grammar plus defaults such as right association (attach low, to the nearest phrase)
1990s to 2000s	Statistics from treebanks	Count how often a verb or noun takes this preposition in a hand parsed corpus, choose the more likely tree
2010s onward	Contextual neural models	Each word vector depends on the whole sentence, so telescope near astronomy and telescope near ship get different representations

Lesson: How machines resolve ambiguity

TIPS AND TRAPS

IN THE EXAM

If an exam asks you to classify an example, ask one question: does any single word have two meanings? If yes, lexical. If every word is clear but the grouping can change, structural.

Lesson: Structural ambiguity: one string, several trees

1.3 Can machines understand language?

Marr's levels, Turing and Total Turing tests, ELIZA against SHRDLU, how NLU systems are built and scored, and the debate on form and meaning.

KEY IDEAS

- **Marr** Computational (what and why), algorithmic (how, representation), implementational (where). Design top down, validate bottom up.
- **Turing test** Operational, text only, not reproducible. Qualia are outside it. The Total Turing Test adds perception and action, seven capabilities.
- **ELIZA and SHRDLU** ELIZA: keyword patterns, no understanding. SHRDLU: world model, reasoning, action and clarifying questions in a blocks world.
- **Fields and pipeline** NLP builds tools, CL models cognition, NLU is the shared goal. Corpus and annotation, model, then validation.
- **Metrics** $P = TP \div (TP + FP)$, $R = TP \div (TP + FN)$, $F1 = 2PR \div (P + R)$. $BLEU = BP \cdot \exp(\Sigma\, w \log p)$. Surprisal $-\log_2 P$. Dependency length.

- **Form against meaning** Firth and Shannon support data. The octopus test, Ettinger, symbol grounding and the Chinese room warn that form is not meaning.

FORMULAS

Precision, recall, F1 Exam favourite

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \mathrm{F1} = \frac{2\mathrm{PR}}{\mathrm{P} + \mathrm{R}}$$

Precision: how much of what you found is right. Recall: how much of the truth you found.

Lesson: NLP, CL and NLU, and how systems are measured

BLEU Exam favourite

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp(\Sigma_n w_n \log p_n)$$

p_n is clipped n-gram precision, BP = 1 if $c > r$, else $e^{1 - r/c}$. Papineni et al. 2002.

Lesson: NLP, CL and NLU, and how systems are measured

Surprisal

$$S(w) = -\log_2 P(w \mid \text{context})$$

Surprising words take longer to read (Hale 2001, Levy 2008).

Lesson: NLP, CL and NLU, and how systems are measured

Dependency length

$$\mathrm{DL} = \Sigma_i |i - \mathrm{head}(i)|$$

Human sentences keep related words close (Gibson 2000).

Lesson: NLP, CL and NLU, and how systems are measured

TABLES

Table: level (marr, 1982) · question · what it specifies

Level (Marr, 1982)	Question	What it specifies
1. Computational	What and why?	The goal of the computation and why it is the right goal
2. Algorithmic or representational	How?	The representation of input and output, and the algorithm that turns one into the other
3. Implementational	Where?	The physical substrate: neurons, gears, silicon

Lesson: Marr's three levels and the robot that flies

Table: · like humans · rationally

	Like humans	Rationally
Thinking	Systems that think like humans (Haugeland, Bellman)	Systems that think rationally (Charniak and McDermott, Winston)
Acting	Systems that act like humans (Kurzweil, Rich and Knight)	Systems that act rationally (Schalkoff, Luger and Stubblefield)

Lesson: The Turing test and what it misses

Table: capability · why

Capability	Why
Natural language processing	Talk with the examiner
Knowledge representation	Store what it is told
Automated reasoning	Answer questions and draw conclusions
Machine learning	Adapt and find patterns
Vision	Recognise the examiner's actions and objects
Motor control	Act on objects when asked
Other senses	Hearing, touch, smell

Lesson: The Turing test and what it misses

Table: · eliza (1966) · shrdlu (about 1970)

	ELIZA (1966)	SHRDLU (about 1970)
Creator	Joseph Weizenbaum, MIT	Terry Winograd, MIT
Pipeline	Language → pattern match → response	Language → world model → reasoning → action → response
Grounding	None, text in and text out	In a simulated micro world
Domain	General chat	Blocks world only
Lesson	Fluent is not understanding	Language can be procedure tied to knowledge and action, at small scale

Lesson: ELIZA and SHRDLU: talking against understanding

Table: field · orientation · goal

Field	Orientation	Goal
NLP	Engineering, application driven	Build tools: search, speech to text, translation, sentiment
Computational linguistics	Science, theory driven	Model the human cognitive system for language
NLU	The shared end goal	A system that really understands, built from description to representation to evaluation

Lesson: NLP, CL and NLU, and how systems are measured

TIPS AND TRAPS

COMMON MISTAKE The professor calls jumping straight to hardware and algorithms "the rush that all computer engineers make". A purely data driven language model is tuned at levels 2 and 3 without a clear level 1 answer to what understanding is.

Lesson: Marr's three levels and the robot that flies

REMEMBER SHRDLU's "OK" matters because it reports that a grounded action was carried out, closing the loop between language and behaviour.

Lesson: ELIZA and SHRDLU: talking against understanding

STEP 2 NLU · Text as data

2.1 Sentences, tokens and vocabularies

The preprocessing pipeline, tokens and types, Punkt sentence splitting, Treebank word tokens, vocabularies and BPE subwords.

KEY IDEAS

- **Pipeline** Sentence segmentation, tokenisation, stemming or lemmatisation, POS tagging, counting.
- **Tokens and types** Token = an occurrence, type = a distinct word. N() and B() in FreqDist. TTR = V ÷ N falls as text grows.
- **Punkt** Unsupervised sentence splitter that learned abbreviations. Kept "Dr. jack" together: 3 sentences in the notebook.
- **Treebank tokens** Punctuation split off, Don't → Do n't, It's → It 's. 32 tokens for the notebook paragraph against 26 from split().
- **Subwords** BPE merges the most frequent adjacent pair again and again, so no word is out of vocabulary.

FORMULAS

Tokens and types Exam favourite

$$N = \text{number of tokens}, \quad V = |\text{vocabulary}| = \text{number of types}$$

FreqDist calls these N() (outcomes) and B() (samples).

Lesson: The preprocessing pipeline

Type token ratio

$$\text{TTR} = \frac{V}{N}$$

Higher means more varied vocabulary. It falls as a text gets longer, so only compare texts of similar length.

Lesson: The preprocessing pipeline

TABLES

Table: step · what it does · example

Step	What it does	Example
Sentence segmentation	Split a paragraph into sentences	"I came. I saw." → ["I came.", "I saw."]
Tokenisation	Split each sentence into tokens (words and punctuation)	"John went to Patna." → John, went, to, Patna, .
Stemming	Chop endings with rules. The stem need not be a word	gaming → game, leaves → leav
Lemmatisation	Map to the dictionary form using the part of speech	children → child, leaves → leaf
POS tagging	Give each token a grammatical category	John/NNP went/VBD to/TO Patna/NNP
Frequency distribution	Count tokens or tags	FreqDist(brown.words()).most_common(10)

Lesson: The preprocessing pipeline

Table: convention · example · why

Convention	Example	Why
Punctuation is its own token	boy. → boy, .	So "boy" and "boy." are one type
Contractions split off	Don't → Do, n't It's → It, 's	n't is the word not, 's can be is or a possessive
Abbreviations stay whole	Dr. stays Dr.	The full stop is part of the word
Only the sentence final stop is split	Jodhpur. → Jodhpur, .	Inner stops belong to abbreviations or numbers

Lesson: Word tokenisation

TIPS AND TRAPS

REMEMBER — Full stops are ambiguous in English: end of sentence, abbreviation, decimal point, or part of an email address. Segmentation is a classification problem for every full stop.

Lesson: Sentence segmentation

IN THE EXAM — Every SRILM file in chapter 6 has one sentence per line with tokens separated by spaces. That format is the result of exactly the segmentation and tokenisation in this chapter.

Lesson: Reading text files in Python

2.2 Stems and lemmas

Inflection and derivation, Porter stemming and the measure m, Snowball and Lancaster, WordNet lemmatisation, stem against lemma, and the precision and recall trade off.

KEY IDEAS

- **Stem against lemma** Stem: rule based chop, may not be a word (leaves → leav). Lemma: dictionary form, uses POS (leaves → leaf, children → child).
- **Porter measure** Word = [C](VC)^m[V]. tree 0, trouble 1, troubles 2. y after a consonant is a vowel.

- **Notebook results** Porter: sportingli, fairli, boy'. Snowball: sport, fair, boy. Exercise list differs only on alumnus.
- **Lemmatiser needs POS** WordNetLemmatizer defaults to noun. loving stays loving, but with pos='v' it becomes love.
- **Errors and effect** Over stemming: university, universe → univers. Under stemming: alumnus and alumni stay apart. Stemming raises recall, may lower precision.

FORMULAS

Vocabulary after normalisation

$$V_{\text{after}} \le V_{\text{before}}, \quad \text{reduction} = 1 - \frac{V_{\text{after}}}{V_{\text{before}}}$$

Merging forms can only lower the number of types. Above, 5 types become 1, a reduction of 80 percent.

Lesson: Words have parts

Why counts improve

$$c(\text{connect*}) = c(\text{connect}) + c(\text{connected}) + c(\text{connecting}) + \dots$$

Evidence that was split over five rare types is pooled into one common type. Chapter 4 shows why fewer, larger counts give better probabilities.

Lesson: Words have parts

Porter's word shape

Exam favourite

$$[\text{C}]\ (\text{VC})^m\ [\text{V}]$$

Square brackets mean optional. m counts how many times a vowel run is followed by a consonant run.

Lesson: Stemming: chop with rules

Examples from Porter (1980)

$$\text{m} = 0\text{: tree, by}$$
$$\text{m} = 1\text{: trouble, oats, ivy}$$
$$\text{m} = 2\text{: troubles, private}$$

tr-ee is C V, so m = 0. tr-ou-bl-e is C V C V, so m = 1.

Lesson: Stemming: chop with rules

Precision and recall

Exam favourite

$$P = \frac{\text{relevant returned}}{\text{all returned}} \qquad R = \frac{\text{relevant returned}}{\text{all relevant}}$$

Both lie between 0 and 1.

Lesson: Stem or lemma: choosing well

F1 score

$$F_1 = \frac{2\,P\,R}{P + R}$$

The harmonic mean. It is high only when both P and R are high.

Lesson: Stem or lemma: choosing well

TABLES

Table: word · morphemes · count

Word	Morphemes	Count
rebuilding	re- (bound prefix, again) + build (free root) + -ing (bound suffix, progressive)	3
unhelpful	un- (bound prefix, not) + help (free root) + -ful (bound suffix, full of)	3
teachers	teach (free root) + -er (bound suffix, one who) + -s (bound suffix, plural)	3
impossibility	im- (bound prefix, not) + possible (free root) + -ity (bound suffix, makes a noun)	3

Lesson: Words have parts

Table: · inflection · derivation

	Inflection	Derivation
What it does	Adds grammar: number, tense, person, comparison	Makes a new word, often a new part of speech
Examples	cat → cats, walk → walked, big → bigger	teach → teacher, happy → happiness, nation → national
Part of speech	Stays the same	Often changes (verb → noun)
Lemma	Same lemma as the base word	A different lemma with its own dictionary entry

Lesson: Words have parts

Table: step · job · example rules · examples

Step	Job	Example rules	Examples
1a	Plurals	SSES → SS, IES → I, SS → SS, S → nothing	caresses → caress, ponies → poni, cats → cat
1b	Past tense and -ing	(m > 0) EED → EE, (*v*) ED → nothing, (*v*) ING → nothing, then tidy up	plastered → plaster, motoring → motor, hopping → hop, sing → sing
1c	Final y	(*v*) Y → I	happy → happi, sky → sky
2	Double suffixes	(m > 0) ATIONAL → ATE, IZATION → IZE, FULNESS → FUL	relational → relate, vietnamization → vietnamize
3	More endings	(m > 0) ICATE → IC, FUL → nothing, NESS → nothing	triplicate → triplic, hopeful → hope, goodness → good
4	Long suffixes	(m > 1) AL, ANCE, ENCE, ER, IC, ABLE, IBLE, ANT, EMENT, MENT, ENT, ION, ISM, ATE, IVE ... → nothing	revival → reviv, adjustable → adjust, adoption → adopt
5	Tidy the end	(m > 1) E → nothing, (m > 1 and double L) → single L	probate → probat, rate → rate, controll → control

Lesson: Inside the Porter algorithm

Table: word · porter · snowball · lancaster

Word	Porter	Snowball	Lancaster
sportingly	sportingli	sport	sport
fairly	fairli	fair	fair
gaming	game	game	gam
generously	gener	generous	gen
organization	organ	organ	org
studies	studi	studi	study
maximum	maximum	maximum	maxim

Lesson: Inside the Porter algorithm

Table: · stem · lemma

	Stem	Lemma
Definition	What is left after rule based suffix stripping	The dictionary (citation) form of the word
Real word	Not always: leaves → leav, studies → studi	Always: leaves → leaf, studies → study
Needs	Only a rule list	A dictionary and the part of speech
Irregular forms	Missed: children, saw, better unchanged	Handled: child, see (verb), good (adjective)
Speed	Very fast	Slower (tagging plus lookup)
Typical use	Search engines, quick text classification	Question answering, chatbots, linguistics

Lesson: Stem or lemma: choosing well

TIPS AND TRAPS

REMEMBER

Hindi, Turkish, Finnish and Tamil pack many more endings into one word than English does. One Turkish verb can have hundreds of forms, so normalisation matters even more for them.

Lesson: Words have parts

IN THE EXAM

In an exam, write the C and V pattern under the letters first, then count VC pairs. Most mistakes come from forgetting that y after a consonant is a vowel.

Lesson: Stemming: chop with rules

COMMON MISTAKE Run a POS tagger first and pass each tag to the lemmatiser. Without the tag, most verbs and adjectives come back unchanged, exactly as in the notebook output above.

Lesson: Lemmatisation: look it up

2.3 Counting words and tags

FreqDist, the Brown corpus and the Penn Treebank with real counts, tagsets and tag mapping, and the first bigram probabilities.

KEY IDEAS

- **FreqDist** N() = tokens (outcomes), B() = types (samples), freq(x) = c(x)/N, unseen count = 0. a b a a c b a gives 3 samples, 7 outcomes, freq(a) = 0.5714.
- **Brown numbers** 1,161,192 tokens, 56,057 types, the = 62,713 (5.40%). 472 distinct tags, top tag NN 152,470.
- **PTB numbers** 100,676 tokens, 12,408 types, comma is most common (4,885), the 4,045 (4.02%). 6,592 -NONE- tokens.
- **Tagsets** 8 traditional, 12 universal, 17 UPOS, 45 PTB, about 87 Brown. Fine maps to coarse: ADV 3,171 = RB + RBR + RBS + WRB.
- **Bigram counts** P(w | prev) = c(prev w) / c(prev). From I am Sam: P(am | I) = 2/3.

FORMULAS

Relative frequency Exam favourite

$$\text{freq}(x) = \frac{c(x)}{N}$$

This is also the maximum likelihood estimate of P(x). Chapter 4 builds language models from exactly this ratio.

Lesson: Counting with FreqDist

Check

$$\sum_x \text{freq}(x) = 1$$

The relative frequencies of all samples always add to one, so they form a probability distribution.

Lesson: Counting with FreqDist

Merging tags adds their counts Exam favourite

$$c(\text{ADV}) = c(\text{RB}) + c(\text{RBR}) + c(\text{RBS}) + c(\text{WRB})$$

Every universal tag count is the sum of the PTB tags mapped to it.

Lesson: How many tags? Tagsets

Notebook numbers

$$3{,}171 = 2{,}822 + 136 + 35 + 178$$

RB does not exist in the universal tagset, so fdist["RB"] gave 0 there.

Lesson: How many tags? Tagsets

Bigram probability by counting Exam favourite

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}\, w_i)}{c(w_{i-1})}$$

The maximum likelihood estimate. Chapter 4 explains why it fails for pairs never seen.

Lesson: Counting pairs: towards language models

Most frequent tag baseline

$$\text{tag}(w) = \text{argmax}_t\, c(w, t)$$

Counting word and tag pairs gives the simplest tagger. Chapter 5 adds context.

Lesson: Counting pairs: towards language models

TABLES

Table: call · meaning · value here

Call	Meaning	Value here
fdist[x]	Count of x. An unseen item gives 0, not an error	fdist['a'] = 4, fdist['d'] = 0
N()	Number of outcomes, which is every token counted	7
B()	Number of samples (bins), which is the distinct types	3
freq(x)	Relative frequency, count of x divided by N	4 ÷ 7 = 0.5714
max()	The most common sample	a
most_common(k)	The k most common samples with counts	[('a', 4), ('b', 2)] for k = 2

Lesson: Counting with FreqDist

Table: · brown corpus · penn treebank (nltk sample)

	Brown corpus	Penn Treebank (NLTK sample)
Source	500 samples of American English printed in 1961, from 15 genres such as news, fiction and science	Wall Street Journal news articles
Built by	Francis and Kučera at Brown University	Marcus, Santorini and Marcinkiewicz at the University of Pennsylvania (1993)
Tokens in NLTK	1,161,192	100,676
Types	56,057	12,408
Most common word	the (62,713 times, 5.40%)	, (4,885 times). the is second with 4,045 (4.02%)
Distinct tags	472 (compound tags counted separately)	46 PTB tags, or 12 universal tags

Lesson: Corpora: Brown and the Penn Treebank

Table: tagset · size · example distinctions

Tagset	Size	Example distinctions
Traditional grammar	8	noun, pronoun, verb, adjective, adverb, preposition, conjunction, interjection
Universal (Petrov et al., 2012)	12	NOUN VERB ADJ ADV PRON DET ADP NUM CONJ PRT . X
Universal Dependencies UPOS	17	adds PROPN, AUX, SCONJ, SYM, INTJ, PUNCT
Penn Treebank	45	NN NNS NNP NNPS, VB VBD VBG VBN VBP VBZ, JJ JJR JJS, RB RBR RBS
Brown	about 87 base tags	plus compound forms such as NN-TL and NP$, which is why NLTK counts 472

Lesson: How many tags? Tagsets

Table: fine tagset (ptb, brown) · coarse tagset (universal)

Fine tagset (PTB, Brown)	Coarse tagset (universal)
More information: singular or plural, past or present	Less information
Harder to tag, because there are more classes to confuse	Easier and more accurate
Useful for parsing and grammar checking	Useful across languages, since the same 12 tags fit most languages

Lesson: How many tags? Tagsets

TIPS AND TRAPS

REMEMBER An item seen once is called a hapax legomenon (Greek for "said once"). In real corpora about half of all types are hapaxes. They are the reason smoothing exists, which is the subject of chapter 4.

Lesson: Counting with FreqDist

IN THE EXAM Exam questions often ask you to "verbally describe how the results differ". Mention size, genre, tagset and the -NONE- tokens, and back each point with one number.

Lesson: Corpora: Brown and the Penn Treebank

STEP 3 NLU · Information and surprise

3.1 Zipf's law and the shape of vocabulary

Zipf's law f = k/r, the Brown check, coverage by harmonic numbers, least effort, Mandelbrot and Heaps, and why random typing is also Zipfian.

KEY IDEAS

- **Zipf's law** f = k/r, f × r ≈ constant, slope −1 on log log axes. f_a/f_b = b/a, so the 50th word is 3 times the 150th.
- **Brown check** the 69,971, of 36,412 (ratio 1.92). f × r stays between about 70,000 and 107,000. Real slope about −1.08.
- **Coverage** Top n share = H_n/H_V, H_n ≈ ln n + 0.5772. V = 50,000: top 10 = 25.7%, top 100 = 45.5%.
- **Least effort** Frequent words are short (law of abbreviation). Language balances speaker effort and listener robustness.
- **Monkey typing** Random keys give a power law with α = 1 − ln(1 − s)/ln M (1.0116 for 27 keys) but no predictability.

FORMULAS

Three dice

$$P(\text{sum} = 3) = \frac{1}{216} \approx 0.005, \quad P(\text{sum} = 10) = \frac{27}{216} = 0.125$$

Counts for sums 3 to 18: 1, 3, 6, 10, 15, 21, 25, 27, 27, 25, 21, 15, 10, 6, 3, 1.

Lesson: Do words have an average frequency?

Words

$$c(\text{the}) = 69{,}971, \quad \text{types seen once} = 16{,}540$$

Brown corpus, lower cased. One word fills about 7% of the text while thousands of words appear once. There is no typical value.

Lesson: Do words have an average frequency?

Zipf's law Exam favourite

$$f = \frac{k}{r} \quad \text{or} \quad f \times r \approx k$$

f is the frequency of the word of rank r and k is a constant for the corpus. The frequency falls as one over the rank.

Lesson: Zipf's law

Ratio rule Exam favourite

$$\frac{f_a}{f_b} = \frac{b}{a}$$

The top word is about twice the second, three times the third. The 50th word is three times the 150th.

Lesson: Zipf's law

Share of the top n words

Exam favourite

$$\text{share}(n) = \frac{H_n}{H_V}, \quad H_n = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n}$$

V is the vocabulary size. Total tokens are k H_V and the top n words have k H_n of them.

Lesson: How much text do the top words cover?

Approximation

$$H_n \approx \ln n + \gamma, \quad \gamma \approx 0.5772$$

γ is the Euler Mascheroni constant. The sum of 1/r grows like the area under 1/x.

Lesson: How much text do the top words cover?

Random typing exponent

Exam favourite

$$\alpha = 1 - \frac{\ln(1 - s)}{\ln M}$$

M letters, space probability s. As M grows α tends to 1 for any fixed s.

Lesson: The Zipf mystery: monkey typing

TABLES

Table: function words · content words

Function words	Content words
the, of, and, to, a, in, that	nouns, verbs, adjectives, adverbs
Short and extremely frequent	Longer and rarer
A closed class: new ones almost never appear	An open class: new words appear all the time
Glue that joins the content words	Carry the meaning and the topic
Little information each, since they are predictable	Much information each, since they are surprising

Lesson: Do words have an average frequency?

Table: rank r · word · frequency f · f × r

Rank r	Word	Frequency f	f × r
1	the	69,971	69,971
2	of	36,412	72,824
3	and	28,853	86,559
10	he	9,548	95,480
50	no	2,139	106,950
150	three	610	91,500
1,000	eight	104	104,000
10,000	incredibly	7	70,000

Lesson: Zipf's law

TIPS AND TRAPS

REMEMBER

Why no bell curve? A normal curve appears when a value is the sum of many small independent effects. Word frequency is not a sum. It comes from people choosing words, and a few grammatical words are chosen in almost every sentence.

Lesson: Do words have an average frequency?

IN THE EXAM

The class question "should the 50th word occur three times as often as the 150th?" has answer yes: $f_{50} \div f_{150} = 150 \div 50 = 3$. On Brown the real ratio is 2,139 ÷ 610 = 3.51, a little steeper, as the slope of −1.08 suggests.

Lesson: Zipf's law

3.2 Surprise, entropy and redundancy

Surprisal, entropy and its bounds, Huffman codes and the source coding limit, Shannon's approximations, context and the 50% redundancy of English.

KEY IDEAS

- **Surprisal** I(x) = $-\log_2 p(x)$. p = 1 gives 0 bits, each halving adds 1 bit. Bits, nats, hartleys.
- **Entropy** H = $-\Sigma\ p \log_2 p$ = expected surprise. $0 \le H \le \log_2 n$. Coin 1 bit, 8 sided die 3 bits, certain event 0.
- **Codes** Fixed length needs $\lceil \log_2 n \rceil$ bits. Huffman merges the two least likely. $H \le \bar{L} < H + 1$. Example 1.9 against H 1.846.
- **Redundancy** R = 1 − H/H_max. English about 2.3 of 4.7 bits per letter, so about 50% redundant. u after q carries 0.02 bits.
- **Limits of H** Single symbol entropy ignores order (ABAB has 1 bit). Language needs conditional entropy: context reduces uncertainty.

FORMULAS

Surprisal of one outcome — Exam favourite

$$I(x) = -\log_2 p(x) = \log_2 \frac{1}{p(x)}$$

Measured in bits. A certain event (p = 1) gives 0 bits. Every halving of the probability adds exactly 1 bit.

Lesson: Information is surprise

Units

$$\log_2 \to \text{bits}, \quad \ln \to \text{nats}, \quad \log_{10} \to \text{hartleys}$$

1 nat = 1.4427 bits and 1 hartley = 3.3219 bits. Changing the base only rescales.

Lesson: Information is surprise

Shannon's entropy — Exam favourite

$$H(X) = -\sum_x p(x) \log_2 p(x) = \mathrm{E}[\, -\log_2 p(X)\,]$$

The average uncertainty, the expected surprise, and the fewest bits per outcome any code can use on average. 0 log 0 is taken as 0.

Lesson: Entropy: average surprise

Bounds — Exam favourite

$$0 \le H(X) \le \log_2 n$$

H = 0 only when one outcome is certain. H = $\log_2$ n only when all n outcomes are equally likely.

Lesson: Entropy: average surprise

Fixed length code

$$\text{bits} = \lceil \log_2 n \rceil$$

20 amino acids need 5 bits, 100 words need 7, 50,000 words need 16.

Lesson: Entropy as the shortest code

Source coding theorem — Exam favourite

$$H(X) \le \bar{L} < H(X) + 1$$

No lossless code can average fewer than H bits per symbol. The best symbol by symbol code, Huffman, is within 1 bit.

Lesson: Entropy as the shortest code

Redundancy — Exam favourite

$$R = 1 - \frac{H}{H_{\max}}$$

R = 0 for random text. R near 1 means most of the text could be predicted and removed.

Lesson: Context and redundancy

Shannon's estimate for English

$$R = 1 - \frac{2.3}{\log_2 26} = 1 - \frac{2.3}{4.70} \approx 51\%$$

With about 8 letters of context each character carries about 2.3 bits, not 5. English is roughly half redundant.

Lesson: Context and redundancy

TABLES

Table: source · probabilities · entropy

Source	Probabilities	Entropy
Fair coin	½, ½	1 bit
Coin with P(heads) = 0.9	0.9, 0.1	0.469 bits
Coin with P(heads) = 0.99	0.99, 0.01	0.081 bits
Fair 8 sided die	⅛ each	3 bits
Fair 6 sided die	⅙ each	2.585 bits
Loaded die	P(6) = 0.5, others 0.1	2.161 bits
Certain event	1, 0	0 bits

Lesson: Entropy: average surprise

Table: order · constraint · shannon's sample

Order	Constraint	Shannon's sample
0	All 27 symbols equally likely	XFOML RXKHRJFFJUJ ZLPWCFWKCYJ
1	English letter frequencies	OCRO HLI RGWR NIMILWIS EU LL
2	Letter pairs match English	ON IE ANTSOUTINYS ARE T INCTORE ST
3	Letter triples match English	IN NO IST LAT WHEY CRATICT FROURE
Words, 2	Word pairs match English	THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER

Lesson: Context and redundancy

Table: context so far · class guesses · choices

Context so far	Class guesses	Choices
A ___	any noun	huge
A SMALL ___	bus, book, kitchen, kid	drastically fewer
A SMALL OBLONG ___	table, desk, board	a surprising word raises uncertainty for a moment
... OBLONG READING ___	lamp	almost certain
... LAMP ON THE D___	desk	one letter removes almost all doubt

Lesson: Context and redundancy

TIPS AND TRAPS

REMEMBER Because these measures are logarithms, independent pieces of evidence simply add. That is the reason Turing, Hartley and Shannon all used logs.

Lesson: Information is surprise

REMEMBER Ths sntnc s stll rdbl. Removing the vowels deletes about 38% of the letters but almost none of the information, because vowels are predictable from the consonants. That is redundancy you can see.

Lesson: Context and redundancy

3.3 Mutual information, KL and the noisy channel

Joint and conditional entropy, the chain rule, mutual information and PMI, KL divergence and cross entropy, and noisy channel decoding.

KEY IDEAS

- **Joint and conditional** $H(Y|X) = \Sigma\, p(x)\, H(Y|X=x) = -\Sigma\, p(x,y) \log p(y|x)$. Chain rule $H(X,Y) = H(X) + H(Y|X)$. Umbrella: 0.971 + 0.621 = 1.592.

- **Mutual information** $I = H(Y) - H(Y|X) = H(X) + H(Y) - H(X,Y) = \Sigma\, p(x,y) \log p(x,y)/(p(x)p(y))$. Symmetric, ≥ 0, 0 iff independent. Umbrella 0.378.
- **PMI** $\log_2 c(xy)N/(c(x)c(y))$. New Delhi 8.64, new car 3.32. Collocations score highest. Rare pairs are over rewarded.
- **KL and cross entropy** $D(p\|q) = \Sigma\, p \log p/q \geq 0$, not symmetric, infinite if $q = 0$ where $p > 0$. $H(p,q) = H(p) + D(p\|q)$.
- **Noisy channel** argmax $p(i)\, p(o|i)$: language model times channel model. acress → actress (0.485). BSC 10%: $C = 0.531$.

FORMULAS

Joint entropy

$$H(X, Y) = -\sum_{x,y} p(x, y) \log_2 p(x, y)$$

The information needed on average to specify both values, like "New" and "Delhi" together.

Lesson: Joint and conditional entropy

Conditional entropy

Exam favourite

$$H(Y \mid X) = \sum_x p(x)\, H(Y \mid X = x) = -\sum_{x,y} p(x, y) \log_2 p(y \mid x)$$

The extra information still needed for Y once X is known. The joint probability goes outside the log, the conditional inside.

Lesson: Joint and conditional entropy

Chain rule

Exam favourite

$$H(X, Y) = H(X) + H(Y \mid X)$$

Specify X, then specify Y given X. For n variables: $H(X_1 \ldots X_n) = \Sigma\, H(X_i \mid X_1 \ldots X_{i-1})$.

Lesson: Joint and conditional entropy

Mutual information

Exam favourite

$$I(X : Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X, Y)$$

The overlap of the two entropy circles, in bits. Many textbooks put a semicolon between X and Y in this notation.

Lesson: Mutual information

Probability form

$$I(X : Y) = \sum_{x,y} p(x, y) \log_2 \frac{p(x, y)}{p(x)\, p(y)}$$

Zero exactly when $p(x, y) = p(x)\, p(y)$ for every pair, which is independence.

Lesson: Mutual information

Pointwise mutual information

Exam favourite

$$\mathrm{PMI}(x, y) = \log_2 \frac{p(x, y)}{p(x)\, p(y)} = \log_2 \frac{c(x\, y) \cdot N}{c(x) \cdot c(y)}$$

Above 0 the words attract, 0 means independent, below 0 they avoid each other. MI is the average of PMI over all pairs.

Lesson: Mutual information

Kullback Leibler divergence

Exam favourite

$$D(p \,\|\, q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}$$

The average number of bits wasted by coding data from p with a code built for q.

Lesson: KL divergence and cross entropy

Cross entropy

Exam favourite

$$H(p, q) = -\sum_x p(x) \log_2 q(x) = H(p) + D(p \,\|\, q)$$

Training a language model with cross entropy loss lowers the KL from the truth to the model, since H(p) is fixed by the data.

Lesson: KL divergence and cross entropy

Noisy channel decoding

Exam favourite

$$\hat{\imath} = \operatorname{argmax}_i p(i \mid o) = \operatorname{argmax}_i p(i) \cdot p(o \mid i)$$

p(i) is the language model (how likely is this input). p(o | i) is the channel model (how likely is this output from it). Used for spelling correction, speech recognition, OCR and translation.

Lesson: The noisy channel

Channel capacity

$$C = \max_{p(X)} I(X : Y)$$

A binary channel that flips 10% of bits has $C = 1 - H(0.1) = 0.531$ bits per use.

Lesson: The noisy channel

TABLES

Table: · collocation · association

	Collocation	Association
Examples	New Delhi, fast food, strong tea, make a decision	new car, doctor and nurse
Can a word be swapped?	No: quick food and powerful tea sound wrong	Yes: new bike, new dress
Meaning	Often not built from the parts	Built from the parts
PMI	Highest	Positive but lower

Lesson: Mutual information

Table: example · d(p ‖ q) · d(q ‖ p)

Example	D(p ‖ q)	D(q ‖ p)
p = (0.5, 0.25, 0.25), q uniform	0.0850 bits	0.0817 bits
City A weather (0.7, 0.2, 0.1) against city B (0.3, 0.3, 0.4)	0.539 bits	0.609 bits
p = (0.5, 0.5, 0), q = (0.4, 0.4, 0.2)	0.322 bits	∞

Lesson: KL divergence and cross entropy

TIPS AND TRAPS

COMMON MISTAKE A student guessed $-\Sigma\, p(x,y) \log p(x|y)$. That is not H(Y|X), but it is a correct formula for H(X|Y): the conditioning just runs the other way. In the umbrella example it gives 0.593 bits against 0.621 for H(Y|X).

Lesson: Joint and conditional entropy

COMMON MISTAKE PMI over rewards rare pairs. Two words seen once, together, score $\log_2 N$, more than New Delhi. Always use a minimum count, or a t-score or log likelihood ratio, alongside it.

Lesson: Mutual information

IN THE EXAM If a model gives probability 0 to something that happens, KL and perplexity become infinite. That single fact is why every n-gram model in chapter 4 is smoothed, and why neural models use a softmax, which never outputs exactly 0.

Lesson: KL divergence and cross entropy

STEP 4 NLU · Language models

4.1 N-gram models and the Markov assumption

The chain rule, the Markov assumption, MLE counting, the start and end tags, and the bigram and unigram activity.

KEY IDEAS

- **Markov assumption** An n-gram looks at n − 1 previous words. Bigram = first order Markov chain. Chain rule is exact, the n-gram is an approximation.
- **MLE** P(w | h) = C(h w) ÷ C(h). Unigram P(w) = C(w) ÷ N. Proof: Lagrange multiplier gives the relative frequency.
- **The tags** <s> is context only, </s> is predicted. Activity corpus: N = 17 + 5 = 22, six predicted types.
- **Activity answers** Bigram S1 = 6/625 = 0.0096 (log −2.01773, PP 2.169), S2 = 0.072. Unigram S1 = 3.3075e-5, S2 = 1.0672e-3.
- **Perplexity N** Use the tokens of the sentence being scored: words + </s> for PP, words for PP1. Never the training N.

FORMULAS

Conditional estimate

Exam favourite

$$\text{m}(\text{jelly} \mid \text{peanut butter and}) = \frac{\text{C}(\text{peanut butter and jelly})}{\text{C}(\text{peanut butter and})}$$

Count the whole four word sequence, then divide by how often the three word history occurred.

Lesson: Why predict the next word?

Chain rule (exact)

Exam favourite

$$\text{P}(w_1 \ldots w_n) = \text{P}(w_1) \cdot \text{P}(w_2 \mid w_1) \cdot \text{P}(w_3 \mid w_1\, w_2) \cdots \text{P}(w_n \mid w_1 \ldots w_{n-1})$$

No approximation yet. Every word is conditioned on everything before it.

Lesson: Why predict the next word?

Markov assumption

Exam favourite

$$\text{P}(w_i \mid w_1 \ldots w_{i-1}) \approx \text{P}(w_i \mid w_{i-n+1} \ldots w_{i-1})$$

The next word depends only on the last n – 1 words. An n-gram has n words: n – 1 of context and one predicted.

Lesson: The Markov assumption

Bigram model

$$\text{P}(w_1 \ldots w_n) \approx \prod_i \text{P}(w_i \mid w_{i-1})$$

A bigram model is a first order Markov chain over words. A trigram model is second order.

Lesson: The Markov assumption

Bigram MLE

Exam favourite

$$\text{P}(w_n \mid w_{n-1}) = \frac{\text{C}(w_{n-1}\, w_n)}{\text{C}(w_{n-1})}$$

Because every occurrence of a word is followed by exactly one token, possibly </s>, the bigram counts in a row add up to the count of the history.

Lesson: Counting: the maximum likelihood estimate

General n-gram MLE

$$\text{P}(w_n \mid w_{n-N+1} \ldots w_{n-1}) = \frac{\text{C}(w_{n-N+1} \ldots w_{n-1}\, w_n)}{\text{C}(w_{n-N+1} \ldots w_{n-1})}$$

Pad each sentence with n – 1 start tags so the first word also has a full history.

Lesson: Counting: the maximum likelihood estimate

Unigram MLE

$$\text{P}(w) = \frac{\text{C}(w)}{N}$$

N counts every token the model must predict: the words and one </s> per sentence, never <s>.

Lesson: Counting: the maximum likelihood estimate

TABLES

Table: model · n · context words · factor for "coffee" in "he put the coffee on the table"

Model	n	Context words	Factor for "coffee" in "He put the coffee on the table"
Unigram	1	0	P(coffee)
Bigram	2	1	P(coffee | the)
Trigram	3	2	P(coffee | put the)
4-gram	4	3	P(coffee | He put the)
5-gram	5	4	P(on | He put the coffee)

Lesson: The Markov assumption

Table: token · <s> · i · sam

Token	<s>	I	Sam	am	do	like	</s>	Words	Words and </s>
Count	5	5	5	2	2	3	5	17	22

Lesson: Counting: the maximum likelihood estimate

TIPS AND TRAPS

COMMON MISTAKE A student first answered "count jelly and divide by the total number of words". That is P(jelly), the unigram probability, which ignores the history. The question asks for the conditional probability, which needs the joint count of the whole sequence.

Lesson: Why predict the next word?

IN THE EXAM In class the answer to "how many context words does a 5-gram have?" came out garbled. The rule is simple: n words in total, the last is predicted, so a 5-gram has 4 context words.

Lesson: The Markov assumption

COMMON MISTAKE Three errors in the answer sheet are worth knowing. It labels unigram factors as P(Sam | <s>), but a unigram has no condition. Its product for S1 lists three numbers and gives 3.25×10^{-5}, which comes from rounding 5/22 to 0.227. The exact value is 3.3075×10^{-5}. And many students used N = 17 or 27, or conditioned the unigram on the previous word.

Lesson: The activity: which sentence is better?

IN THE EXAM PP divides the total surprise by words plus </s>, PP1 by words only, so PP1 is always the larger. The N in PP = $P^{-1/N}$ is the length of the sentence being scored, 6 for S1, not the 22 tokens of the training corpus. The professor saw that mistake in many submissions.

Lesson: The activity: which sentence is better?

4.2 Perplexity and evaluating language models

Perplexity from entropy, ppl and ppl1, file perplexity, underflow, the WSJ table and fair comparison.

KEY IDEAS

- **Perplexity** PP = $2^{\hat{H}} = P^{-1/N}$. Uniform over k words gives PP = k. "He put the coffee on": 13.288 bits, $\hat{H}$ 2.658, PP 6.31.
- **ppl and ppl1** Same logprob. ppl divides by words − OOV − zeroprobs + sentences, ppl1 by words − OOV − zeroprobs. File ppl from totals, not an average.
- **Evaluation** Extrinsic (task) against intrinsic (likelihood, log likelihood, perplexity). Use logs to avoid underflow below about 10^{-308}.
- **Rules** Never train or tune on the test set. Compare perplexities only with the same test text and vocabulary. Report OOVs.
- **WSJ** Unigram 962, bigram 170, trigram 109 (9.91, 7.41, 6.77 bits). Test perplexity is 2 to the cross entropy, which is at least the true entropy.

FORMULAS

Pointwise conditional entropy

$$-\log_2 \mathrm{m}(w_i \mid h_i)$$

The surprise of one word given its history.

Lesson: From surprise to perplexity

Total surprise

$$H_{\text{total}} = -\sum_i \log_2 \mathrm{m}(w_i \mid h_i) = -\log_2 \mathrm{m}(w_1 \ldots w_N)$$

Logs turn the chain rule product into a sum.

Lesson: From surprise to perplexity

Perplexity Exam favourite

$$\mathrm{PP} = 2^{\hat{H}}, \quad \hat{H} = \frac{H_{\text{total}}}{N}, \quad \text{so} \quad \mathrm{PP} = \mathrm{m}(w_1 \ldots w_N)^{-1/N}$$

Lower perplexity means the model found the text less surprising.

Lesson: From surprise to perplexity

ppl Exam favourite

$$\text{ppl} = 10^{-\text{logprob} \div (\text{words} - \text{OOVs} - \text{zeroprobs} + \text{sentences})}$$

Counts one </s> per sentence: how hard is it to predict every token, including where the sentence ends?

Lesson: PP, PP1 and whole files

ppl1 Exam favourite

$$\text{ppl1} = 10^{-\text{logprob} \div (\text{words} - \text{OOVs} - \text{zeroprobs})}$$

Words only. The cost of predicting the end is charged to the words, so ppl1 ≥ ppl always.

Lesson: PP, PP1 and whole files

The three intrinsic measures Exam favourite

$$\text{likelihood } \mathrm{m}(\text{test}), \quad \text{log likelihood } \log \mathrm{m}(\text{test}), \quad \text{PP} = \mathrm{m}(\text{test})^{-1/\mathrm{N}}$$

Higher likelihood, higher log likelihood and lower perplexity all mean the same thing, on the same test set and vocabulary.

Lesson: Evaluating language models

Test set cross entropy Exam favourite

$$\hat{H} = -\frac{1}{N} \log_2 \mathrm{m}(w_1 \ldots w_N) \approx H(p, \mathrm{m}) = H(p) + D(p \,\|\, \mathrm{m})$$

For long text this tends to the cross entropy of the truth p with the model m.

Lesson: Perplexity as cross entropy

TABLES

Table: · extrinsic · intrinsic

	Extrinsic	Intrinsic
What	Put the model inside an application and measure the application	Measure the model alone on held out text
Examples	The London word order task, translation, speech recognition, spelling correction	Likelihood, log likelihood, perplexity on a test set
Strength	Measures what we really care about	Cheap, fast and repeatable
Weakness	Slow and expensive	A lower perplexity usually, but not always, gives a better application

Lesson: Evaluating language models

Table: model · perplexity · bits per word

Model	Perplexity	Bits per word
Unigram	962	9.91
Bigram	170	7.41
Trigram	109	6.77

Lesson: Evaluating language models

TIPS AND TRAPS

REMEMBER Base 2 gives bits, base e gives nats, base 10 gives hartleys. Perplexity does not depend on the base as long as you raise the same base you took the log with: $2^{\hat{H} \text{ in bits}} = 10^{\hat{H} \text{ in hartleys}}$. That is why SRILM can print $\log_{10}$ values and still match the textbook.

Lesson: From surprise to perplexity

IN THE EXAM For long documents PP and PP1 converge, since there is one end tag per twenty or more words. For three word toy sentences they differ a lot (1.93 against 2.40). In an exam, say which N you used and never compare PP from one model with PP1 from another.

Lesson: PP, PP1 and whole files

REMEMBER Adding another source of information, such as tone of voice or a facial expression, can only lower conditional entropy on average: H(Y | X, Z) ≤ H(Y | X). Sarcasm is the classic case where the text misleads and the voice resolves it.

Lesson: Perplexity as cross entropy

4.3 Unseen words and smoothing

<UNK>, Laplace and add-k, discounting methods and Kneser-Ney, backoff, interpolation and λ tuning.

KEY IDEAS

- **Two zeros** OOV word: map to <UNK>. Unseen n-gram of known words: smooth, back off or interpolate. One zero factor gives P = 0 and PP = ∞.
- **Laplace and add-k** (C + k) ÷ (C(h) + kV). V = 7 for the activity. S3 Laplace bigram = 1/17010, PP 5.07. Sums to 1 by adding kV.
- **Discounting** Absolute: max(C − d, 0)/C(h) + λP(w), λ = d N_{1+}/C(h). Good-Turing: c* = (c+1)N_{c+1}/N_c, unseen N_1/N. KN uses continuation counts.
- **Backoff** Use the highest order with evidence, discounted, with α so the row sums to 1. Problem: "John eats blue", unseen is not always sparse.
- **Interpolation** Σλ P = 1 when Σλ = 1. Tune λ on dev (S3: best near 0.48, PP 6.43), report test once. EM for many λ.

FORMULAS

Laplace (add-one) Exam favourite

$$P_{\text{Lap}}(w) = \frac{C(w) + 1}{N + V}, \quad P_{\text{Lap}}(w \mid h) = \frac{C(h\,w) + 1}{C(h) + V}$$

V counts the types the model can predict: here 7, with </s> and <UNK> but not <s>.

Lesson: <UNK>, Laplace and add-k

Add-k

$$P(w \mid h) = \frac{C(h\,w) + k}{C(h) + kV}$$

A smaller k, such as 0.5 or 0.01, steals less from the seen words. Choose k on a dev set.

Lesson: <UNK>, Laplace and add-k

Absolute discounting Exam favourite

$$P(w \mid h) = \frac{\max(C(h\,w) - d,\, 0)}{C(h)} + \lambda(h)\, P(w), \quad \lambda(h) = \frac{d \cdot N_{1+}(h\,\bullet)}{C(h)}$$

Subtract a fixed d of about 0.75 from every seen count. $N_{1+}(h\,\bullet)$ is the number of different words seen after h.

Lesson: Better smoothing: discounting

Good-Turing

$$c^* = (c + 1)\,\frac{N_{c+1}}{N_c}, \quad P(\text{unseen}) = \frac{N_1}{N}$$

N_c = number of types seen exactly c times. Things seen once tell you how much is still unseen.

Lesson: Better smoothing: discounting

Witten-Bell

$$\lambda(h) = \frac{N_{1+}(h\,\bullet)}{N_{1+}(h\,\bullet) + C(h)}$$

A history followed by many different words is likely to be followed by a new one.

Lesson: Better smoothing: discounting

Kneser-Ney Exam favourite

$$P_{cont}(w) = \frac{|\{v : C(v\ w) > 0\}|}{\text{number of bigram types}}$$

Absolute discounting, but the lower order model counts how many different words come before w, not how often w occurs.

Lesson: Better smoothing: discounting

Backoff (Katz 1987)

$$P_{BO}(w \mid h) = P^*(w \mid h) \text{ if } C(h\ w) > 0, \text{ else } \alpha(h)\ P_{BO}(w \mid \text{shorter } h)$$

Non linear: use the highest order with evidence. P* is discounted and α hands exactly the freed mass to the lower order, so the result sums to 1.

Lesson: Backoff, interpolation and choosing λ

Linear interpolation Exam favourite

$$\hat{P}(w \mid h) = \lambda_3\, P(w \mid u\ v) + \lambda_2\, P(w \mid v) + \lambda_1\, P(w), \quad \Sigma\, \lambda = 1, \lambda \geq 0$$

Always mix all levels. Proof of normalisation: $\Sigma_w\, \hat{P} = \lambda_3 \cdot 1 + \lambda_2 \cdot 1 + \lambda_1 \cdot 1 = 1$.

Lesson: Backoff, interpolation and choosing λ

TABLES

Table: · out of vocabulary word · unseen n-gram of known words

	Out of vocabulary word	Unseen n-gram of known words
Example	"linguistics" was never seen at all	"am do": both words were seen, never together
Even the unigram is	0	fine
Fix	Map it to an <UNK> token	Smoothing, backoff or interpolation

Lesson: Two kinds of zero

Table: λ · 0.1 · 0.3 · 0.5

λ	0.1	0.3	0.5	0.7	0.9
dev PP	7.248	6.605	6.438	6.814	8.924

Lesson: Backoff, interpolation and choosing λ

TIPS AND TRAPS

COMMON MISTAKE The answer sheet got this one wrong twice. It wrote P(linguistics | like) = 0.01 instead of 1/10. Then it multiplied in the unigram values 0.034 and 0.207 in brackets, which is neither a Laplace bigram nor a proper backoff, and does not sum to 1. SRILM's vocabulary also contains <s> (its file lists 8 unigrams), so its add-one numbers can differ slightly. In the exam, state your V.

Lesson: <UNK>, Laplace and add-k

IN THE EXAM Chen and Goodman (1998) compared all of these. Modified Kneser-Ney with interpolation was the most accurate classic n-gram smoother, and it is still the default in toolkits such as KenLM. In assignment 2 it also won on the Brown corpus.

Lesson: Better smoothing: discounting

4.4 The SRILM toolkit and assignment 2

SRILM commands, count and ARPA files, test output, and the Brown corpus results of assignment 2.

KEY IDEAS

- **Workflow** ngram-count -write (counts), ngram-count -lm (model), ngram -lm -ppl -debug 2 (scores). Add -unk to map unknown words.

- **ARPA** $\log_{10}$ probabilities and backoff weights. −99 = log 0. Toy header ngram 1=8, ngram 2=14. $\log_{10}(5/22)$ = −0.6434527.
- **Default smoothing** Good-Turing with Katz backoff. Flags: -addsmooth k, -cdiscount, -wbdiscount, -kndiscount, -interpolate.
- **Brown results** Bigram ppl: add-one 2493, GT 642, WB 397, absolute 392, KN 352. Trigram lowers all except add-one.
- **Stop words and tuning** Removing punctuation and stop words raises ppl (397, 611, 3235), a trend only. Best k 0.001, λ 0.7, dev 581, test 681.

FORMULAS

Flags for smoothing

-addsmooth k, -cdiscount d, -wbdiscount, -kndiscount, -interpolate

Add-k, absolute discounting, Witten-Bell, Kneser-Ney, and making a method interpolated instead of backoff.

Lesson: SRILM in three commands

Default

no flag = Good-Turing discounting with Katz backoff

Heard as "good viewing" in the lecture recording.

Lesson: SRILM in three commands

TABLES

Table: step · command · what it does

Step	Command	What it does
1. Count	`ngram-count -text train.txt -order 2 -write train.count`	Writes every n-gram with its count
2. Estimate	`ngram-count -text train.txt -order 2 -lm train.lm -addsmooth 0`	Turns counts into probabilities and backoff weights, saved as an ARPA file
3. Test	`ngram -lm train.lm -ppl test.txt -debug 2`	Prints each word's probability, then logprob, ppl and ppl1

Lesson: SRILM in three commands

Table: arpa value · check by hand

ARPA value	Check by hand
−0.6434527 for I, Sam and </s>	$\log_{10}(5/22)$
−1.041393 for am and do	$\log_{10}(2/22)$
−0.8653014 for like	$\log_{10}(3/22)$
−0.2218488 for <s> Sam and Sam I	$\log_{10}(3/5)$
−0.1760913 for like </s>	$\log_{10}(2/3)$
−99 for <s>	The ARPA stand in for log 0. <s> is never predicted.

Lesson: SRILM in three commands

Table: srilm option · bigram ppl · bigram ppl1 · trigram ppl

SRILM option	Bigram ppl	Bigram ppl1	Trigram ppl	Trigram ppl1	$\log_{10}$ P(<unk>)
-addsmooth 1	2493.24	3600.57	3620.71	5321.27	−5.428
Absolute discounting	391.85	518.76	384.91	509.14	−1.401
Good-Turing	642.33	870.35	629.24	851.77	−5.408
Witten-Bell	396.83	525.66	385.06	509.35	−1.298
Kneser-Ney	352.45	464.28	340.18	447.38	−0.878

Lesson: Assignment 2, question by question

Table: data version · test words · model unigrams · ppl

Data version	Test words	Model unigrams	ppl	ppl1
Original	305,056	40,607	396.83	525.66
Punctuation tokens removed	268,236	40,590	610.78	859.35
Punctuation and stop words removed	148,503	40,262	3,234.52	7,019.32

Lesson: Assignment 2, question by question

Table: -addsmooth k · 1 · 0.1 · 0.01

-addsmooth k	1	0.1	0.01	0.001	0.0001
Best λ	0.3	0.6	0.7	0.7	0.6
Dev ppl	1123.66	833.51	656.01	581.03	582.25

Lesson: Assignment 2, question by question

TIPS AND TRAPS

REMEMBER If SRILM cannot be installed, NLTK's nltk.lm module (MLE, Laplace, KneserNeyInterpolated) reproduces the toy numbers. Check its padding and its token count for perplexity against SRILM's ppl and ppl1 before trusting it.

Lesson: SRILM in three commands

REMEMBER The professor asked students to do the coding activities with their own hands and not with a chatbot: "knowing is one thing, doing it with your hand is a different experience". The widgets in this chapter show the results so you can check yours, not replace them.

Lesson: From the toy corpus to the Brown corpus

STEP 5 NLU · Tagging and syntax

5.1 Part of speech tagging

The tagging task, tagsets, ambiguity, look up and backoff taggers, and generative against discriminative models.

KEY IDEAS

- **The task** Give each word its part of speech. Two cues: the word itself and its context. book: noun or verb. can: modal, noun or verb.
- **Tagsets** Traditional about 8, universal 12, UPOS 17, Penn Treebank 45, Brown about 87. More tags, harder but more informative.
- **Ambiguity numbers** About 11% of Brown types but 40% of tokens are ambiguous. Most frequent tag baseline about 92%, good taggers 97% and above.
- **Kinds of tagger** Look up with backoff, generative HMM (P(W, T)), discriminative MaxEnt, perceptron, CRF (P(T | W)), neural BiLSTM and BERT.
- **Generative and discriminative** argmax P(T | W) = argmax P(W, T) because P(W) is fixed. Not the same axis as supervised and unsupervised.

FORMULAS

The problem statement

Exam favourite

$$\text{given words } W = w_1 \dots w_n, \quad \text{find tags } \hat{T} = \operatorname{argmax}_T \mathrm{P}(T \mid W)$$

Supervised: train on sentences whose correct tags were added by linguists (a gold standard), then tag new sentences whose tags are hidden. It is classification into one of q tags, done for a whole sequence so the choices interact.

Lesson: What part of speech tagging is

Generative

Exam favourite

$$\hat{T} = \operatorname{argmax}_T \mathrm{P}(W, T) = \operatorname{argmax}_T \mathrm{P}(T) \cdot \mathrm{P}(W \mid T)$$

Learns how tag sequences go and which words each tag produces: a noun tends to produce student, dog, book. "A joint win for both parties."

Lesson: Generative or discriminative

Discriminative

Exam favourite

$$\hat{T} = \operatorname{argmax}_T \mathrm{P}(T \mid W)$$

Learns directly which tags fit these words in this context: reads in "The student reads books" is likely a verb.

Lesson: Generative or discriminative

TABLES

Table: sentence · word · tag · why

Sentence	Word	Tag	Why
I read a book	book	noun	a thing you read
Please book a ticket	book	verb	an action
She can dance	can	modal verb	ability
A can of soda	can	noun	a container
They can fish	can	verb (or modal)	they put fish in cans, or they are able to fish

Lesson: What part of speech tagging is

Table: input · british · left · waffles

Input	British	left	waffles	on	Falkland	Islands
Reading 1: the political Left is indecisive	N	N	V	P	N	N
Reading 2: British people left some waffles	N	V	N	P	N	N

Lesson: What part of speech tagging is

Table: tagset · tags · examples

Tagset	Tags	Examples
Traditional grammar	about 8	noun, pronoun, verb, adjective, adverb, preposition, conjunction, interjection
Universal (Petrov and colleagues, 2012)	12	NOUN, VERB, ADJ, ADV, PRON, DET, ADP, NUM, CONJ, PRT, ".", X
Universal Dependencies UPOS	17	adds PROPN, AUX, SCONJ, SYM, INTJ and others
Penn Treebank	45 (46 in NLTK's list)	NN, NNS, NNP, NNPS, VB, VBD, VBG, VBN, VBP, VBZ, JJ, JJR, JJS, ...
Brown corpus	about 87	even finer, with separate tags for forms of be and have

Lesson: How many tags are there?

Table: family · examples · idea

Family	Examples	Idea
Rule based and look up	Regular expression tagger, default tagger, unigram and bigram taggers in NLTK	Hand rules such as "-ing → VBG", or a table of the commonest tag, backing off when unseen
Statistical, generative	HMM tagger	Model P(W, T) = P(T) P(W | T): tags generate words. Decode with Viterbi.
Statistical, discriminative	Maximum entropy, perceptron, CRF	Model P(T | W) directly from rich features: prefix, suffix, capital letters, neighbouring words and tags
Neural	BiLSTM CRF, BERT, spaCy, Stanza	Learn dense embeddings and read context in both directions

Lesson: Kinds of tagger

TIPS AND TRAPS

REMEMBER Not every ambiguity is a tagging problem. In "The fisherman sat near the bank", bank is a noun in both readings, river bank and money bank. That is word sense ambiguity, a different task.

Lesson: What part of speech tagging is

REMEMBER Whatever tagger you use, do error analysis. Find where it fails, group the errors, then improve. The professor asked students to do activity 3 with their own hands, not with a chatbot: "You will only learn when you do it yourself."

Lesson: Kinds of tagger

5.2 Finite state machines and the HMM

FSAs, the HMM five tuple, the joint probability, validity checks, training by counting and enumeration.

KEY IDEAS

- **FSA** Five tuple (Q, Σ, q0, δ, F). Accept only if every symbol has an arrow and the machine ends in a final state. Sheep talk is /baa+!/.
- **HMM** Five tuple (Q, A, O, B, π). Tags are hidden, words are observed. A probabilistic Moore machine: states emit words.
- **Joint probability** P(W, T) = π(t1) b(w1 | t1) × product over i ≥ 2 of a(t(i−1) → ti) b(wi | ti). 2n factors. Markov and output independence assumptions.
- **Validity** Each row of A, each row of B and π sums to 1. A missing cell is 1 minus the rest of its row.
- **Training** Count starts, tag bigrams and (tag, word) pairs, then divide by row totals. Zeros need smoothing.

FORMULAS

The FSA five tuple Exam favourite

$$(Q, \Sigma, q_0, \delta, F)$$

Q is the set of states, Σ the alphabet of symbols, q_0 the start state, δ the transition function (state, symbol) → state, and F the final states, drawn as double circles.

Lesson: Finite state automata

The HMM five tuple Exam favourite

$$(Q, A, O, B, \pi)$$

Q the tag states, A the transition table P(tag | previous tag), O the vocabulary of words, B the emission table P(word | tag), π the start probabilities.

Lesson: The hidden Markov model

Joint probability of words and tags Exam favourite

$$P(W, T) = \pi(t_1)\, b_{t_1}(w_1) \prod_{i=2}^{n} a_{t_{i-1}t_i}\, b_{t_i}(w_i)$$

One start, n emissions and n − 1 transitions: 2n factors in all. "We multiply the transition probability with the emission probability" at every position.

Lesson: The hidden Markov model

Start

$$\pi(t) = \frac{\text{C(sentences starting with } t)}{\text{number of sentences}}$$

Lesson: Training an HMM is counting

Transition Exam favourite

$$a(t \to u) = \frac{\text{C}(t\ u)}{\text{C}(t \text{ followed by any tag})}$$

The tag bigram MLE.

Lesson: Training an HMM is counting

Emission Exam favourite

$$b(w \mid t) = \frac{\text{C}(w \text{ tagged } t)}{\text{C}(t)}$$

A unigram model of words for each tag.

Lesson: Training an HMM is counting

TABLES

Table: input · path · result · reason

Input	Path	Result	Reason
cat	q0 → q1 → q2 → q3	accept	ends in q3, a final state
car	q0 → q1 → q2 → q3	accept	the arrow q2 r q3 exists
ca	q0 → q1 → q2	reject	q2 is not final
cut	q0 → q1, stuck	reject	q1 has no arrow for u
cart	q0 → … → q3, stuck	reject	q3 has no arrow for t

Lesson: Finite state automata

Table: machine · output is attached to · in the hmm

Machine	Output is attached to	In the HMM
Mealy	each arrow (transition)	not used
Moore	each state	yes: a tag state emits a word

Lesson: The hidden Markov model

Table: · to a · to n · sum

	to A	to N	Sum
π (start)	1/2	1/2	1
from A	1/3	2/3	1
from N	1/10	9/10	1

Lesson: Is this HMM valid?

Table: emits · crazy · killer · clown

Emits	crazy	killer	clown	problem	Sum
A	1	0	0	0	1
N	0	1/3	1/3	1/3	1

Lesson: Is this HMM valid?

Table: tags · π · b(killer) · a · b(clown) · p(w, t)

Tags	π · b(killer) · a · b(clown)	P(W, T)
A A	0.25 × 0 × 0 × 0	0
A N	0.25 × 0 × 1.0 × 0.4	0
N A	0.75 × 0.3 × 0.5 × 0	0
N N	0.75 × 0.3 × 0.5 × 0.4	**0.045**

Lesson: Scoring every tag sequence

TIPS AND TRAPS

REMEMBER How does this help with language? Finite state machines run tokenisers, regular expression taggers, spelling checkers and morphology (leaves maps to leaf + N + PL). Put probabilities on the arrows and you get a Markov chain, the n-gram model of step 4. Let each state also emit a word and you get the HMM.

Lesson: Finite state automata

COMMON MISTAKE Do not confuse an HMM with a bigram word model. In the bigram model the thing you condition on, the previous word, is visible. In the HMM it is a hidden tag, so to find tags you must search (Viterbi) or sum (forward) over them.

Lesson: The hidden Markov model

IN THE EXAM Exam questions often hide one cell. Fill it with 1 minus the rest of its row (problem G2). The rows are what sum to 1, never the columns: P(N | A) + P(N | N) = 2/3 + 9/10 is not 1 and does not need to be.

Lesson: Is this HMM valid?

COMMON MISTAKE Two different killer clown HMMs appear in the deck. Slides 23 and 24 give P(killer clown, N N) = 1/2 × 1/3 × 9/10 × 1/3 = 0.05. Slide 27 gives 0.045. Same winner, different number. Always use the tables the question gives you (problem G4).

Lesson: Scoring every tag sequence

5.3 Decoding with Viterbi

Brute force against dynamic programming, the Viterbi and forward recursions, posteriors, ties and trigram HMMs.

KEY IDEAS

- **Search size** Brute force scores q^T sequences. Viterbi does about T × q² steps. PTB 45 tags, 25 words: 2.1 × 10^41 against 50,625.
- **Viterbi recursion** v1(j) = πj bj(w1). vt(j) = max over i of vt−1(i) aij bj(wt). Keep back pointers, take the best last cell and trace back.
- **Forward** Same table with sum: P(W) = sum of the last column. Posterior P(T | W) = P(W, T) ÷ P(W). they can fish: 0.073164 and 0.7233.
- **Why it works** Principle of optimality: under the Markov assumption the best path passes through the best partial path into each cell.
- **Ties and trigrams** Ties: pick either, or return k best. Trigram HMM (TnT): O(T q³), needs smoothing, about 96.7% on PTB.

FORMULAS

The decoding objective

Exam favourite

$$\hat{T} = \operatorname{argmax}_{t_1 \ldots t_n} \prod_i P(w_i \mid t_i) \cdot P(t_i \mid t_{i-1})$$

The first factor is the emission, the second the transition. $P(t_1 \mid t_0)$ means the start probability $\pi(t_1)$.

Lesson: Why brute force fails

Viterbi recursion

Exam favourite

$$v_1(j) = \pi_j \, b_j(w_1) \qquad v_t(j) = \max_i v_{t-1}(i) \cdot a_{ij} \cdot b_j(w_t)$$

$v_t(j)$ is the probability of the best tag path that ends in tag j after reading t words. Store the winning i as a back pointer.

Lesson: The Viterbi algorithm

Cost

$$O(T \cdot q^2) \text{ time}, \quad O(T \cdot q) \text{ memory}$$

For each of T positions and q tags, compare q predecessors.

Lesson: The Viterbi algorithm

Forward recursion

Exam favourite

$$\alpha_1(j) = \pi_j \, b_j(w_1) \qquad \alpha_t(j) = \Sigma_i \, \alpha_{t-1}(i) \cdot a_{ij} \cdot b_j(w_t)$$

$\alpha_t(j)$ is the total probability of the first t words with tag j at position t.

Lesson: The forward algorithm and confidence

Sentence probability and posterior

$$P(W) = \Sigma_j \, \alpha_T(j) \qquad P(T \mid W) = \frac{P(W, T)}{P(W)}$$

The posterior says how sure the model is of one tagging.

Lesson: The forward algorithm and confidence

Second order HMM

$$P(t_i \mid t_{i-2}, t_{i-1}) \qquad \text{cost } O(T \cdot q^3)$$

Viterbi now runs over pairs of tags. There are q^3 transition parameters instead of q^2, so sparsity grows and interpolation is needed, exactly as for trigram language models.

Lesson: Ties, trigrams and real taggers

TABLES

Table: q (tags) · t (words) · sequences q t · viterbi steps t × q²

q (tags)	T (words)	Sequences q^T	Viterbi steps T × q²
2	4	16	16
12	10	6.19×10^{10}	1,440
45	25	2.14×10^{41}	50,625
87	10	2.48×10^{19}	75,690

Lesson: Why brute force fails

Table: π · n 0.7 · m 0.1 · v 0.2

π	N 0.7	M 0.1	V 0.2

Lesson: The Viterbi algorithm

Table: a (from ↓, to →) · n · m · v

A (from ↓, to →)	N	M	V
N	0.2	0.4	0.4
M	0.1	0	0.9
V	0.6	0.1	0.3

Lesson: The Viterbi algorithm

Table: b · they · can · fish

B	they	can	fish
N	0.3	0.2	0.5
M	0	1.0	0
V	0	0.3	0.7

Lesson: The Viterbi algorithm

Table: tags · p(w, t) · p(t | w)

Tags	P(W, T)	P(T | W)
N M V	0.052920	0.7233
N V N	0.007560	0.1033
N V V	0.005292	0.0723
N M N	0.004200	0.0574
N N V	0.002352	0.0321
N N N	0.000840	0.0115

Lesson: The forward algorithm and confidence

TIPS AND TRAPS

IN THE EXAM

Exam recipe. 1: write π, A and B as tables and check the rows. 2: if T ≤ 3 and q ≤ 2, enumerating all paths is fine, show every product. 3: otherwise fill a q × T table column by column, writing the winning predecessor in each cell. 4: back trace from the best last cell. 5: state the tag sequence and its probability. Strike out zero emissions first.

Lesson: The Viterbi algorithm

5.4 Chunking and named entities

Chunks, regex chunkers, IOB and BIOES encoding, NER models and entity level F1.

KEY IDEAS

- **Chunk** A flat, non overlapping group of words such as [the yellow dog]. Chunking is shallow parsing. Constituents nest, chunks do not.
- **IOB** B-X begins, I-X continues, O is outside. 2k + 1 labels. Turns span finding into tagging with the same models.
- **Decoding IOB** A span is a B-X plus the following I-X tags. An I-X after O or another type is repaired as B-X.
- **NER** PER, ORG, LOC, DATE, MONEY. Perceptron, CRF, BiLSTM CRF and BERT. Features: capitals, shape, gazetteers, neighbours.
- **Scoring** Entity level precision, recall and F1 with exact start, end and type. Token accuracy is inflated by O.

FORMULAS

IOB tags Exam favourite

$$\textbf{B-}X \text{ begins a chunk of type } X \quad \textbf{I-}X \text{ is inside it} \quad \textbf{O} \text{ is outside every chunk}$$

With k chunk types there are 2k + 1 labels. The input can be the words and their POS tags.

Lesson: IOB: spans as tags

Entity level F1 Exam favourite

$$P = \frac{\text{correct spans}}{\text{predicted spans}} \quad R = \frac{\text{correct spans}}{\text{gold spans}} \quad F1 = \frac{2PR}{P + R}$$

A predicted span counts only if its start, end and type all match. Token accuracy is misleading because most tokens are O.

Lesson: Named entity tagging

TABLES

Table: · chunk · constituent

	Chunk	Constituent
Shape	flat, never nested	nested inside a full tree
Example	[the yellow dog]	[[the yellow dog] [in [the park]]]
Defined by	any useful group, often base noun and verb phrases	grammar: noun phrase, verb phrase, adjective phrase
Cost	fast and robust, a tagging problem	slower, needs a parser (chapter 5.5)

Lesson: What a chunk is

Table: type · examples

Type	Examples
PER (person)	Sidharth Ranjan, Marie Curie
ORG (organisation)	United, IIT Jodhpur, the World Health Organization
LOC or GPE (place)	San Francisco, the Falkland Islands, India
DATE, MONEY, PERCENT	next Tuesday, 500 rupees, 12%

Lesson: Named entity tagging

TIPS AND TRAPS

REMEMBER This is why POS tagging comes first. The chunker never looks at the words, only at the tags, so tagging errors flow straight into chunking errors.

Lesson: What a chunk is

REMEMBER Why B at all? Two chunks of the same type can sit side by side: "gave [the girl] [a book]". With only I and O, the girl a book would merge into one span. B marks where the second one starts.

Lesson: IOB: spans as tags

5.5 Constituency and dependency syntax

Constituency tests, CFGs, dependency trees, PCFGs, attachment ambiguity and Catalan growth.

KEY IDEAS

- **Constituents** Groups that act as one unit. Tests: movement, substitution by a pro form, fragment answer, coordination, clefting.
- **CFG** Four tuple (N, Σ, R, S). Rules A → β. Bracket notation stores trees. Penn Treebank supplies trees and rule counts.

- **Dependencies** Head → dependent arcs with labels (nsubj, obj, det). A tree: one head per word, n arcs with ROOT, no cycles. UD makes content words heads.
- **PCFG** P(tree) = product of rule probabilities, rules for each non terminal sum to 1. Elephant: VP attach 0.01575 against 0.0105, ratio 1.5.
- **Ambiguity growth** k PPs give Catalan C(k + 1) parses: 2, 5, 14, 42. CKY parses in O(n³).

FORMULAS

Bracket notation

Exam favourite

$$(S\ (NP\ (N\ Fed))\ (VP\ (V\ raises)\ (NP\ (N\ interest)\ (N\ rates))))$$

Each opening bracket starts a node and names its label. Treebanks store trees this way, on one line, called a linearised tree.

Lesson: Constituents and phrase structure

CFG four tuple

Exam favourite

$$(N, \Sigma, R, S)$$

N non terminals (S, NP, VP ...), Σ terminals (the words), R rules of the form A → β, and the start symbol S. A sentence is grammatical if S can be rewritten into it, and the rewriting history is its parse tree.

Lesson: Context free grammars

Probabilistic CFG

Exam favourite

$$P(\text{tree}) = \prod_{\text{rules } A \to \beta \text{ used}} P(\beta \mid A), \quad \text{with } \Sigma_\beta\, P(\beta \mid A) = 1 \text{ for each A}$$

Rule probabilities come from treebank counts: P(VP → V NP) = C(VP → V NP) ÷ C(VP). Choose the tree with the higher product.

Lesson: Attachment ambiguity and PCFGs

Catalan growth

$$C_n = \frac{(2n)!}{(n+1)!\,n!} \quad \text{parses with } k \text{ PPs} = C_{k+1}\text{: } 2, 5, 14, 42, 132 \ldots$$

Exponential growth, just like tag sequences. Chart parsers (CKY) share sub trees in O(n³) time.

Lesson: Attachment ambiguity and PCFGs

TABLES

Table: test · what you do · "about drugs" in "john talked to the children about drugs"

Test	What you do	"about drugs" in "John talked to the children about drugs"
Movement	Move the group as a block	John talked [about drugs] [to the children]: fine
Fragment answer	Use it alone as an answer	What did John talk about? About drugs.
Substitution	Replace it with a pro form (it, there, did so)	John talked to the children about it.
Coordination	Join it with a similar group	about drugs and about school
Clefting	"It was X that ..."	It was about drugs that John talked to the children.

Lesson: Constituents and phrase structure

Table: rule · reads as

Rule	Reads as
S → NP VP	a sentence is a noun phrase followed by a verb phrase
NP → Det N	a noun phrase can be a determiner and a noun
VP → V NP | V	a verb phrase is a verb with or without an object
Det → the, N → dog | cat, V → chased | slept	the lexicon: which words fill each slot

Lesson: Context free grammars

Table: kind · labels · meaning

Kind	Labels	Meaning
Argument	nsubj, obj, iobj	the participants the verb needs: who paints, what is painted, to whom
Modifier	det, amod, nmod, advmod	optional extras: determiners, adjectives, noun and adverb modifiers

Lesson: Dependency structure

Table: · dependency structure · phrase structure

	Dependency structure	Phrase structure
Shows	head to dependent relations, with function labels	nested phrases with category labels (NP, VP)
Nodes	only the words	words plus phrase nodes
Good for	free word order, relation extraction, who did what to whom	phrase boundaries, coordination, grammar engineering
Treebank	Universal Dependencies, 100+ languages	Penn Treebank

Lesson: Dependency structure

Table: tool · what it gives you

Tool	What it gives you
NLTK	corpora (Brown, Penn Treebank sample), n-gram and perceptron taggers, chunkers, CFG and PCFG parsers
spaCy	fast pipelines: tokens, POS, lemmas, dependency parse, named entities
Stanza	neural pipeline for 60+ languages with UPOS, XPOS, UD dependency parsing and NER

Lesson: Attachment ambiguity and PCFGs

TIPS AND TRAPS

IN THE EXAM — A student asked whether this is a DAG, as in compilers. It is stricter: a tree. Every word has exactly one head, there are no cycles, and a sentence of n words has exactly n arcs counting the ROOT arc.

Lesson: Dependency structure

REMEMBER — In spaCy, every token knows its text, t.pos_, t.dep_ and t.head.text. Running it on the elephant sentence shows which attachment the trained parser picked.

Lesson: Attachment ambiguity and PCFGs

STEP 6 NLU · Exam revision

6.1 Exam revision: from school to research universities

Exam style questions from school level to research universities, with worked answers, a formula sheet and a mock quiz.

FORMULAS

Type token ratio — Exam favourite

$$\text{TTR} = \frac{\text{number of types}}{\text{number of tokens}}$$

Types are distinct words. Tokens are running words.

Lesson: Tier 1: school and first year

Zipf's law — Exam favourite

$$f \times r \approx C$$

The word of rank r appears about C ÷ r times.

Lesson: Tier 1: school and first year

Entropy

$$H = -\Sigma\, p \log_2 p$$

Average surprise in bits. A fair coin has 1 bit.

Lesson: Tier 1: school and first year

TABLES

Table: tier · typical format · what earns the marks · where to practise

Tier	Typical format	What earns the marks	Where to practise
School and first year	Short answers, counting, small probabilities	Clear definitions with an example, careful counting	Lesson 2
University semester and IIT course exams	Numericals by hand, short notes, compare and contrast	Formula first, every step shown, the final answer stated with its meaning	Lesson 3
NPTEL and GATE style papers	Multiple choice, multiple select and numerical answer type, no partial credit	Speed, recognising the trap option, exact rounding	Lesson 4
MIT, Stanford and Cambridge style	Derivations, proofs, design and critique	Assumptions named, a complete argument, when it breaks	Lesson 5

Lesson: How NLP is examined

Table: training corpus

Training corpus
<s> I am Sam </s>
<s> Sam I am </s>
<s> Sam I like </s>
<s> Sam I do like </s>
<s> do I like Sam </s>

Lesson: Tier 2: university semester and IIT course exams

Table: shortcut · use it for

Shortcut	Use it for
Uniform over k outcomes: H = $\log_2$ k, PP = k	Instant entropy and perplexity of a uniform model
PP = 2^H = $10^{-\log 10\ P \div N}$	Switching between bits and SRILM's base 10
n-gram table size V^n	Why trigrams are sparse
IOB labels 2k + 1, BIOES 4k + 1	Label counts for k entity types
Brute force q^T, Viterbi T q^2, trigram HMM T q^3	Complexity questions
Dependency tree: n words, n arcs with ROOT	Counting arcs
KL ≥ 0, equal to 0 only when P = Q	True or false on divergences
H(X, Y) = H(X) + H(Y | X)	Chain rule for entropy

Lesson: Tier 3: multiple choice and numerical answer papers

Table: topic · formula

Topic	Formula
Zipf	$f \times r \approx C$
Surprisal, entropy	$I(x) = -\log_2 p(x)$, $H = -\Sigma\, p \log_2 p$
Conditional entropy, chain rule	$H(Y \mid X) = H(X, Y) - H(X)$
Mutual information	$I(X : Y) = H(X) - H(X \mid Y)$, $PMI = \log_2 P(x, y) \div P(x)\, P(y)$
KL, cross entropy	$KL(P \parallel Q) = \Sigma\, P \log_2(P \div Q)$, $H(P, Q) = H(P) + KL$
Bigram MLE	$P(w \mid h) = C(h\ w) \div C(h)$
Laplace, add-k	$(C(h\ w) + 1) \div (C(h) + V)$, $(C(h\ w) + k) \div (C(h) + kV)$
Interpolation	$\lambda_3 P(w \mid u\ v) + \lambda_2 P(w \mid v) + \lambda_1 P(w)$, λs sum to 1
Perplexity	$P^{-1/N} = 2^H$, SRILM $ppl = 10^{-lp \div (words - OOV + sents)}$
HMM joint	$\pi(t_1)\, b(w_1 \mid t_1) \prod a(t_{i-1} \to t_i)\, b(w_i \mid t_i)$
Viterbi, forward	$v_t(j) = \max_i v_{t-1}(i)\, a_{ij}\, b_j(w_t)$, α with Σ
Entity F1	$2PR \div (P + R)$ on exact spans
PCFG	$P(tree) = \prod P(rule)$
Catalan	$C_n = (2n)! \div ((n + 1)!\, n!)$

Lesson: Formula sheet and last minute checks

TIPS AND TRAPS

COMMON MISTAKE Do not mix tokens and types. "How many words" is ambiguous in an exam. Say which you counted.

Lesson: Tier 1: school and first year

COMMON MISTAKE State your convention for <s> and </s> before computing. Counting </s> as a predicted token changes both the probability and the N in perplexity.

Lesson: Tier 2: university semester and IIT course exams

IN THE EXAM Trap options to expect: counting <s> as a predicted token, mixing log base 2 and 10, forgetting to add V in the Laplace denominator, treating columns of an HMM table as summing to 1, and calling a chunk a constituent.

Lesson: Tier 3: multiple choice and numerical answer papers

IN THE EXAM In derivations, name each assumption when you use it: Markov, output independence, a fixed vocabulary, independent sentences. Examiners often ask what breaks when one fails.

Lesson: Tier 4: derivations, design and critique

STEP 1 NLP · Neurons and learning

1.1 From regression to learning by gradient descent

Weights, bias, squared error, gradient descent, learning rate, partial derivatives and the chain rule.

KEY IDEAS

- **Model and cost** A model has knobs (weights w and bias b). The squared error cost $C = \sum(\hat{y} - y)^2$ scores a guess. For the lecture data w = 1, b = 1.37 gives 0.67.
- **Gradient descent** Move each knob against its slope: $\theta \leftarrow \theta - \eta\, \partial C/\partial\theta$. Too large η overshoots or diverges, too small η is slow.

- **One knob example** NN() = b, cost (b − 4)², slope 2(b − 4). The distance to 4 is multiplied by 1 − 2η each step, η = 0.5 lands in one step.
- **Partial derivatives** ∂C/∂w = ∑2(wx + b − y)x and ∂C/∂b = ∑2(wx + b − y). The cost over (w, b) is a bowl with one minimum.
- **Chain rule** For ŷ = tanh(wx + b): ∂L/∂w = 2(ŷ − y)(1 − ŷ²)x. A negative gradient raises w, but a large step can still raise the cost.

FORMULAS

Linear model

$$\text{wags}(\text{pats}) = w \cdot \text{pats} + b$$

w is the weight (how steep), *b* is the bias (where the line starts at pats = 0).

Lesson: A model is a guess with knobs

Squared error cost

Exam favourite

$$C(w, b) = \sum_i (wx_i + b - y_i)^2$$

Squaring makes every error positive and punishes big errors much more than small ones.

Lesson: A cost function scores a guess

Mean squared error

$$\text{MSE} = \frac{1}{n} \sum (\hat{y}_i - y_i)^2$$

The same cost divided by the number of examples, so it does not grow with the data size.

Lesson: A cost function scores a guess

Slope of the teaching cost

$$d\,C \div db = 2(b - 4)$$

Positive slope: move left. Negative slope: move right. Zero: you are at the bottom.

Lesson: Rolling downhill: gradient descent

Gradient descent update

Exam favourite

$$b \leftarrow b - \eta \cdot \frac{dC}{db}$$

η is the learning rate, the fraction of the slope we step by.

Lesson: Rolling downhill: gradient descent

Partial derivatives of the squared error

Exam favourite

$$\frac{\partial C}{\partial w} = \sum 2(wx + b - y) \cdot x \qquad \frac{\partial C}{\partial b} = \sum 2(wx + b - y)$$

Hold the other knob fixed and take the ordinary derivative.

Lesson: Two knobs: partial derivatives and the cost bowl

Update both at once

$$w \leftarrow w - \eta\, \partial C/\partial w, \quad b \leftarrow b - \eta\, \partial C/\partial b$$

The vector (∂C/∂*w*, ∂C/∂*b*) is the gradient. It points uphill, so we subtract it.

Lesson: Two knobs: partial derivatives and the cost bowl

Chain rule for one neuron

Exam favourite

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w} = 2(\hat{y} - y) \cdot (1 - \hat{y}^2) \cdot x$$

z = *wx* + *b*. The derivative of tanh z is 1 − tanh² z.

Lesson: The exam question: a tanh model by the chain rule

TABLES

Table: guess · predictions · cost

Guess	Predictions	Cost
w = 0.25, *b* = 0.68	0.93, 1.18, 1.68	20.12
w = 1.00, *b* = 1.37	2.37, 3.37, 5.37	0.67
w = 13/14, *b* = 1.5 (best)	2.43, 3.36, 5.21	0.643

Lesson: A cost function scores a guess

Table: learning rate η · factor 1 − 2η · what happens

Learning rate η	Factor 1 − 2η	What happens
0.02	0.96	Very slow, many steps
0.1	0.8	Smooth and steady
0.5	0	Lands on 4 in one step
0.9	−0.8	Jumps across, oscillates, settles
1.05	−1.1	Each jump grows: diverges

Lesson: Rolling downhill: gradient descent

TIPS AND TRAPS

REMEMBER The cost is a function of the knobs *w* and *b*, not of the data. The data is fixed. That is why we can draw it as a curve or a surface over the knobs.
Lesson: A cost function scores a guess

COMMON MISTAKE A negative slope does not by itself promise a lower cost after the step. If η is too large the step overshoots and the cost can go up. The lecture asked exactly this.
Lesson: Rolling downhill: gradient descent

IN THE EXAM Expect "if ∂C/∂w is negative, does w increase or decrease after the update?" It increases, because w − η × (negative) adds a positive amount.
Lesson: Rolling downhill: gradient descent

REMEMBER Derivatives matter because they are the exact slope. No guessing is needed, and the same rule works for models with billions of knobs.
Lesson: Two knobs: partial derivatives and the cost bowl

IN THE EXAM Formulas for derivatives are given in the exam if you need them. If none is given, the question can be answered without one. This was said in class.
Lesson: The exam question: a tanh model by the chain rule

1.2 One neuron, the sigmoid and backpropagation

A neuron, the sigmoid, the decision line, backpropagation and the flowers notebook.

KEY IDEAS

- **Neuron** NN(x) = σ(w·x + b) with σ(z) = 1 ÷ (1 + e^{-z}). NN(2, 1) with w = (0.5, 0.2), b = 0.3 is 0.82. Output above 0.5 means class 1.
- **Boundary** σ(z) = 0.5 exactly at z = 0, so one neuron draws a straight line. It cannot separate XOR.
- **Chain rule** ∂cost/∂w_1 = 2(pred − target) · σ(z)(1 − σ(z)) · x_1. For b the last factor is 1. Update w ← w − η · gradient.
- **Notebook** 10,000 random single flower steps with η = 0.1 from random weights. The mystery flower (4.5, 1) comes out 0.99, red.
- **Loss choice** Squared error with a sigmoid learns slowly when confidently wrong. Cross entropy gives the clean gradient (σ − y)x.

FORMULAS

One neuron with two inputs

Exam favourite

$$\mathrm{NN}(m_1, m_2) = \sigma(w_1 m_1 + w_2 m_2 + b)$$

Multiply each input by its weight, add the bias, then squash with the sigmoid σ.

Lesson: A neuron: weigh, add, squash

Sigmoid

Exam favourite

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Maps any real number into (0, 1). σ(0) = 0.5.

Lesson: A neuron: weigh, add, squash

Decision rule

$$\text{class 1 if } \sigma(z) \geq 0.5, \text{ which is the same as } z \geq 0$$

The boundary is the line w_1·length + w_2·width + b = 0.

Lesson: Why the sigmoid, and where to draw the line

Inverse of the sigmoid (logit)

$$z = \ln \frac{p}{1 - p}$$

p = 0.8 needs z = ln 4 = 1.386.

Lesson: Why the sigmoid, and where to draw the line

Symmetry

$$\sigma(-z) = 1 - \sigma(z)$$

Used again in negative sampling.

Lesson: Why the sigmoid, and where to draw the line

The three links

Exam favourite

$$\frac{\partial \text{cost}}{\partial w_1} = \frac{\partial \text{cost}}{\partial \text{pred}} \cdot \frac{\partial \text{pred}}{\partial z} \cdot \frac{\partial z}{\partial w_1} = 2(\text{pred} - \text{target}) \cdot \sigma'(z) \cdot \text{length}$$

For w_2 the last link is width, for b it is 1.

Lesson: Backpropagation is the chain rule

Slope of the sigmoid

Exam favourite

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Largest at z = 0, where it is 0.25. Called sigmoid_p in the notebook.

Lesson: Backpropagation is the chain rule

TABLES

Table: length · 3 · 2 · 4

length	3	2	4	3	3.5	2	5.5	1	4.5
width	1.5	1	1.5	1	0.5	0.5	1	1	1
type	1	0	1	0	1	0	1	0	?

Lesson: A neuron: weigh, add, squash

Table: output · reading

Output	Reading
0.92	very likely red
0.5	cannot tell, on the boundary
0.08	very likely blue

Lesson: Why the sigmoid, and where to draw the line

TIPS AND TRAPS

REMEMBER — The 0.5 threshold is a choice, not a law. A cancer screening test may use 0.2 to miss fewer cases. Changing the threshold moves the decision line without retraining.

Lesson: Why the sigmoid, and where to draw the line

COMMON MISTAKE Do not forget the σ'(z) link. Without it you are differentiating a straight line, not a neuron.

Lesson: Backpropagation is the chain rule

IN THE EXAM A typical question gives $\hat{y} = \sigma(wx + b)$ or $\tanh(wx + b)$, one data point and the cost, and asks for $\partial L/\partial w$, the predicted value, and the direction w moves.

Lesson: Backpropagation is the chain rule

1.3 Activation functions: why networks need a bend

Why networks need non-linear activations, and how the classic and modern ones compare.

KEY IDEAS

- **Why** Without activations $W_2W_1x = Wx$: depth adds nothing. Non-linearities bend space and make networks universal function approximators.
- **Classic units** Threshold (no gradient), sigmoid (0, 1), tanh = $2\sigma(2z) - 1$ in (−1, 1), hard tanh. Sigmoid and tanh saturate.
- **ReLU** max(z, 0). Slope 1 for $z > 0$ and 0 for $z < 0$ (dead neurons). Leaky and parametric ReLU keep a small negative slope.
- **Smooth gates** swish = $z\cdot\sigma(z)$, GELU = $z\cdot\Phi(z) \approx z\cdot\sigma(1.702z)$, used in BERT and RoBERTa.
- **Trade off** More non-linear layers mean more expressive power but harder optimisation and overfitting risk.

FORMULAS

Linear layers collapse Exam favourite

$$W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x} = W\mathbf{x}$$

Ten linear layers are still one linear layer, with ten times the compute.

Lesson: Why a network needs a bend

A layer with an activation

$$\mathbf{h} = f(W\mathbf{x} + \mathbf{b})$$

f is applied to each number separately. That one bend is what gives depth its power.

Lesson: Why a network needs a bend

tanh from the sigmoid Exam favourite

$$\tanh(z) = 2\sigma(2z) - 1$$

Twice as steep at 0 (slope 1 against 0.25) and centred on zero.

Lesson: Threshold, sigmoid, tanh and hard tanh

Slopes

$$\sigma' = \sigma(1 - \sigma) \quad \tanh' = 1 - \tanh^2$$

Both shrink toward 0 for large |z|: the saturation problem.

Lesson: Threshold, sigmoid, tanh and hard tanh

Hard tanh

$$\text{hardtanh}(z) = -1 \text{ for } z < -1, \text{ z in between, } 1 \text{ for } z > 1$$

Straight pieces instead of curves, so no exponentials are needed.

Lesson: Threshold, sigmoid, tanh and hard tanh

ReLU and leaky ReLU Exam favourite

$$\text{ReLU}(z) = \max(z, 0) \quad \text{leaky}(z) = z \text{ for } z > 0,\ a\cdot z \text{ otherwise}$$

Leaky ReLU keeps a small slope a (for example 0.01). Parametric ReLU learns a.

Lesson: ReLU and its smooth successors

Swish

$$\text{swish}(z) = z \cdot \sigma(z)$$

A soft gate between 0 and 1 multiplies the input. Ramachandran and others, arXiv 1710.05941.

Lesson: ReLU and its smooth successors

GELU Exam favourite

$$\text{GELU}(z) = z \cdot P(X \le z),\ X \sim N(0, 1) \approx z \cdot \sigma(1.702 z)$$

Scales z by how likely a standard normal is to be below it. Used in BERT and RoBERTa. arXiv 1606.08415.

Lesson: ReLU and its smooth successors

TABLES

Table: unit · range · good · problem

Unit	Range	Good	Problem
Threshold	0 or 1	Simple	Slope 0, no learning
Sigmoid	(0, 1)	Probability reading	Saturates, outputs always positive
tanh	(−1, 1)	Zero centred, faster convergence	Saturates, needs exponentials
Hard tanh	[−1, 1]	Cheap to compute	Flat outside [−1, 1]

Lesson: Threshold, sigmoid, tanh and hard tanh

Table: z · −2 · −0.5 · 0

z	−2	−0.5	0	0.5	2
ReLU	0	0	0	0.5	2
leaky (a = 0.01)	−0.02	−0.005	0	0.5	2
swish	−0.238	−0.189	0	0.311	1.762
GELU (approx.)	−0.064	−0.150	0	0.350	1.936

Lesson: ReLU and its smooth successors

TIPS AND TRAPS

COMMON MISTAKE More non-linear layers give more expressive power but also harder optimisation and a higher risk of overfitting. The slide closes with exactly this warning.

Lesson: Why a network needs a bend

REMEMBER Sigmoid outputs are always positive, so the gradients of all weights into the next neuron share one sign. Updates zig-zag. tanh avoids this, which is why it was preferred in hidden layers.

Lesson: Threshold, sigmoid, tanh and hard tanh

IN THE EXAM Expect "what is the gradient of ReLU for x < 0?" Answer 0. Expect also "why tanh over sigmoid in hidden layers?" Answer: zero centred and stronger slope.

Lesson: ReLU and its smooth successors

STEP 2 NLP · Word vectors

2.1 From symbols to word vectors

One-hot vectors, distributional semantics, context windows, dense embeddings and cosine similarity.

KEY IDEAS

- **One-hot** Length = vocabulary size (500,000+), one 1. Every pair is orthogonal (dot 0, distance √2), so one-hot vectors carry no similarity.
- **Distributional semantics** A word's meaning comes from the words near it (Firth 1957). Context = words within ±m positions.
- **Word vectors** Dense, short, learned so that words in similar contexts get similar vectors. Also called embeddings. A distributed representation.
- **Similarity** Dot product and cosine = a·b ÷ (‖a‖‖b‖). banking against monetary: cosine 0.857.
- **WordNet fails** Incomplete synonym lists, similarity is graded not binary, and context changes meaning.

FORMULAS

One-hot vector

$$\text{motel} = [0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0] \quad \text{hotel} = [0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0]$$

A localist representation: one position means one word.

Lesson: Words as symbols: the one-hot problem

Every pair is orthogonal Exam favourite

$$\text{hotel} \cdot \text{motel} = 0 \quad \text{and} \quad \|\text{hotel} - \text{motel}\| = \sqrt{2}$$

All different words are equally far apart, so the vectors carry no similarity at all.

Lesson: Words as symbols: the one-hot problem

Context of a word

$$\text{context}(w) = \text{the words within a window of } \pm m \text{ positions around each occurrence of } w$$

Many occurrences give many contexts. Together they describe the word.

Lesson: You shall know a word by the company it keeps

Dot product Exam favourite

$$\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$$

Large when the vectors point the same way. Word2vec uses it directly as a similarity score.

Lesson: Dense word vectors and cosine similarity

Cosine similarity Exam favourite

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}$$

1 for the same direction, 0 for unrelated, −1 for opposite. It ignores vector length.

Lesson: Dense word vectors and cosine similarity

TABLES

Table: left context · word · right context

Left context	Word	Right context
problems turning into	**banking**	crises as happened
Europe needs unified	**banking**	regulation to replace
given its	**banking**	system a shot

Lesson: You shall know a word by the company it keeps

TIPS AND TRAPS

REMEMBER One-hot vectors are not useless. They are how a word enters a network: multiplying a one-hot vector by an embedding matrix simply picks out one row. That lookup is the first step of word2vec.

Lesson: Words as symbols: the one-hot problem

COMMON MISTAKE Cosine similarity is not a probability and can be negative. Do not compare a cosine with a probability.

Lesson: Dense word vectors and cosine similarity

2.2 Word2vec: skip-gram with the full softmax

Skip-gram objective, softmax prediction, gradients, SGD, the notebook, CBOW and parameter counts.

KEY IDEAS

- **Word2vec idea** Slide a window over the text. Use the centre word to predict each outside word, and adjust the vectors to make true neighbours probable (Mikolov et al. 2013).
- **Objective** $J(\theta) = -(1/T) \sum_t \sum_j \log P(w_{t+j} \mid w_t)$: log, minus sign, average.

- **Prediction** $P(o \mid c) = \exp(u_o \cdot v_c) \div \sum_w \exp(u_w \cdot v_c)$: dot product, exp, normalise. Two vectors per word, two roles.
- **Gradient** $\partial J/\partial v_c = -u_o + \sum P(w \mid c) u_w$, observed minus expected. Trained with SGD, one window at a time.
- **Variants and size** Skip-gram predicts context from centre, CBOW centre from averaged context. Full softmax has 2Vd parameters: 6,000,000 for V = 10,000, d = 300.

FORMULAS

Likelihood of the whole text

$$L(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m,\, j \neq 0} P(w_{t+j} \mid w_t, \theta)$$

θ is every word vector. The product runs over every position and every neighbour in the window.

Lesson: The word2vec idea: predict the neighbours

Objective to minimise

Exam favourite

$$J(\theta) = -\frac{1}{T} \sum_t \sum_j \log P(w_{t+j} \mid w_t, \theta)$$

The average negative log likelihood.

Lesson: The word2vec idea: predict the neighbours

Skip-gram prediction function

Exam favourite

$$P(o \mid c) = \frac{\exp(u_o \cdot v_c)}{\sum_{w \in V} \exp(u_w \cdot v_c)}$$

Step 1 dot product (similarity), step 2 exp (always positive), step 3 divide by the sum (a distribution).

Lesson: The prediction function: dot, exponentiate, normalise

Softmax

$$\text{softmax}(x)_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

"Max" because large scores dominate, "soft" because small ones still get some probability.

Lesson: The prediction function: dot, exponentiate, normalise

Gradient for the centre vector

Exam favourite

$$\frac{\partial J}{\partial v_c} = -u_o + \sum_{w \in V} P(w \mid c)\, u_w$$

Observed context vector against the expected context vector under the model.

Lesson: Training: the gradient and the notebook loop

In the notebook's matrix form

$$\text{error} = p - y, \quad \partial J/\partial h = W \cdot \text{error}, \quad \partial J/\partial W = h \otimes \text{error}$$

y is the one-hot vector of the true context word. $\otimes$ is the outer product.

Lesson: Training: the gradient and the notebook loop

Stochastic gradient descent

$$\theta \leftarrow \theta - \eta \nabla_\theta J_{\text{window}}(\theta)$$

Update after each sampled window, not after the whole corpus.

Lesson: Training: the gradient and the notebook loop

CBOW input

$$h = \frac{1}{2m} \sum_{-m \le j \le m,\, j \neq 0} v_{w_{t+j}}$$

Average the context vectors, then predict the centre word with the same softmax.

Lesson: Skip-gram, CBOW and why two vectors

Parameters of skip-gram with full softmax

Exam favourite

$$|E| + |W| = V \cdot d + d \cdot V = 2Vd$$

V = 10,000 and d = 300 give 6,000,000 numbers.

Lesson: Skip-gram, CBOW and why two vectors

TABLES

Table: step · what it does · why

Step	What it does	Why
1. Dot product $u_o \cdot v_c$	Scores how compatible the two words are	banking and crises score high, banking and skillet low
2. Exponentiate	Makes every score positive	Probabilities cannot be negative, and big scores grow much bigger
3. Normalise	Divides by the sum over the vocabulary	The probabilities add to 1

Lesson: The prediction function: dot, exponentiate, normalise

Table: model · predicts · learns

Model	Predicts	Learns
Skip-gram	the context words from the centre word	vectors that capture co-occurrence patterns, good for rare words
CBOW	the centre word from the bag of context words	similar vectors, faster to train
GloVe (chapter 2.4)	log co-occurrence counts	vectors that encode global count statistics

Lesson: Skip-gram, CBOW and why two vectors

Table: loss · cost per pair · note

Loss	Cost per pair	Note
Naive softmax	V dot products	Simple but expensive with many output classes
Hierarchical softmax	about $\log_2 V$	A binary tree over the vocabulary
Negative sampling	1 + K sigmoids	The standard choice, chapter 2.3

Lesson: Skip-gram, CBOW and why two vectors

TIPS AND TRAPS

IN THE EXAM — Sigmoid or softmax? The lecture answer: sigmoid for a yes or no (binary) decision, softmax for picking one of many classes. Full word2vec picks one word out of V, so it uses softmax. Negative sampling turns it into many yes or no decisions, so it uses sigmoids.

Lesson: The prediction function: dot, exponentiate, normalise

REMEMBER — J depends on every window in the corpus, possibly billions. Computing the full gradient before one update would take forever, so all neural NLP models use stochastic or mini batch gradient descent.

Lesson: Training: the gradient and the notebook loop

COMMON MISTAKE — With full softmax the error vector p − y has V entries, so every single update touches all V output vectors. That is the cost negative sampling removes.

Lesson: Training: the gradient and the notebook loop

2.3 Skip-gram with negative sampling

Negative sampling objective, gradients, the 3/4 noise distribution, sigmoid counting and the notebook.

KEY IDEAS

- **Why** The softmax denominator sums over the whole vocabulary. Negative sampling replaces it with 1 + K binary logistic regressions.
- **Objective** $J_{neg} = -\log \sigma(u_o \cdot v_c) - \sum_k \log \sigma(-u_k \cdot v_c)$, $\sigma(-x) = 1 - \sigma(x)$.

- **Noise distribution** $P_n(w) \propto U(w)^{3/4}$: frequent words like "the" drop, rare words rise. The true context is never used as noise.
- **Counting question** Sigmoids per centre word = (1 + K) × number of context words: window 2, K = 10 gives 44.
- **Notebook** K = 3, counts$^{0.75}$, 500 epochs: cats and dogs become nearest neighbours (cosine 0.98). Optimum: u·v = PMI − log K.

FORMULAS

The costly part

$$P(o \mid c) = \frac{\exp(u_o \cdot v_c)}{\sum_{w \in V} \exp(u_w \cdot v_c)}$$

The denominator sums over all words in the vocabulary.

Lesson: The expensive denominator

Loss for one pair

Exam favourite

$$J_{neg}(o, v_c, U) = -\log \sigma(u_o \cdot v_c) - \sum_{k=1}^{K} \log \sigma(-u_k \cdot v_c)$$

First term: make the real pair likely. Second: make each noise pair unlikely.

Lesson: The negative sampling objective

Symmetry of the sigmoid

$$\sigma(-x) = 1 - \sigma(x)$$

So $\log \sigma(-u_k \cdot v_c) = \log(1 - \sigma(u_k \cdot v_c))$.

Lesson: The negative sampling objective

Gradients

Exam favourite

$$\frac{\partial J}{\partial v_c} = (\sigma(u_o \cdot v_c) - 1)u_o + \sum_k \sigma(u_k \cdot v_c)u_k, \quad \frac{\partial J}{\partial u_k} = \sigma(u_k \cdot v_c)v_c$$

Each term is (prediction − label) × the other vector. Only 1 + K output vectors change.

Lesson: The negative sampling objective

Noise distribution

Exam favourite

$$P_n(w) = \frac{U(w)^{3/4}}{\sum_{w'} U(w')^{3/4}}$$

α = 3/4 was found to work best. α = 1 is the raw unigram, α = 0 is uniform.

Lesson: Which words to use as noise: U(w) to the 3/4

TABLES

Table: pair · label · training pushes

Pair	Label	Training pushes
(cat, sat)	real, 1	cat and sat closer
(cat, the)	real, 1	closer
(cat, apple)	noise, 0	cat and apple apart
(cat, car)	noise, 0	apart
(cat, quickly)	noise, 0	apart

Lesson: The expensive denominator

Table: word · count · u(w) · p n (w)

Word	Count	U(w)	$P_n(w)$
the	1000	0.6002	0.5409
of	600	0.3601	0.3687
bank	40	0.0240	0.0484
river	20	0.0120	0.0288
skillet	5	0.0030	0.0102
aardvark	1	0.0006	0.0030

Lesson: Which words to use as noise: U(w) to the 3/4

Table: notebook run · nearest to cats · nearest to dogs

Notebook run	Nearest to cats	Nearest to dogs
Full softmax	milk 0.577, dogs 0.483	balls 0.634, cats 0.483
Negative sampling, K = 3	dogs 0.979, milk 0.888	cats 0.979, balls 0.817

Lesson: The negative sampling notebook

TIPS AND TRAPS

IN THE EXAM — Count positives and negatives per context word, then multiply by the number of context words. Read whether the question gives K per pair or per centre word.

Lesson: The negative sampling objective

REMEMBER — Stop words such as "the" are still used as positive pairs when they appear in the window. They are only down weighted as negatives. This was asked in class.

Lesson: The negative sampling notebook

2.4 Co-occurrence counts, SVD and GloVe

Co-occurrence matrices, SVD, GloVe ratios, the GloVe loss and weighting function, and the notebook.

KEY IDEAS

- **Co-occurrence matrix** X_{ij} counts word j inside the window of word i. Window matrices give word space, word by document matrices give topics (LSA).
- **SVD** $X = U\Sigma V^T$, keep the top k singular values for k-dimensional vectors. Raw counts need log, caps or removing function words first.
- **GloVe idea** Meaning is in ratios P(k | i) ÷ P(k | j): much more than 1, much less than 1, or about 1 (ice, steam, solid, gas, water, fashion).
- **GloVe loss** $J = \sum f(X_{ij})(w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$, $f(x) = (x \div x_{max})^{\alpha}$ below x_{max} else 1, $x_{max} = 100$, $\alpha = 3/4$.
- **Notebook** Window 2 with 1 ÷ distance, d = 20, $x_{max} = 10$, final vectors w + w̃. 882 parameters for 80 pairs, so the loss hits 0 and the tiny model memorises.

FORMULAS

Window co-occurrence

Exam favourite

$$X_{ij} = \text{number of times word j appears within m words of word i}$$

A symmetric window counts left and right neighbours. Window 1 on the slide, 5 to 10 is more common.

Lesson: Count the neighbours directly

Word vector by counting

$$\text{vector}(i) = \text{row i of } X$$

Its length is the vocabulary size V, so the vector grows with the vocabulary.

Lesson: Count the neighbours directly

SVD

Exam favourite

$$X = U \Sigma V^T$$

U and V are orthonormal: unit columns at right angles. Σ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$.

Lesson: Shrink the matrix with SVD

Truncated SVD

Exam favourite

$$X_k = U_k \Sigma_k V_k^T$$

Keep the top k singular values. Word i gets the vector row i of $U_k\Sigma_k$.

Lesson: Shrink the matrix with SVD

Singular values

$$\sigma_i = \sqrt{\lambda_i(X^T X)}$$

The square roots of the eigenvalues of $X^T X$.

Lesson: Shrink the matrix with SVD

Kept energy

$$\frac{\sigma_1^2 + \ldots + \sigma_k^2}{\sum \sigma_i^2}$$

The share of the squared size of X that the rank k copy keeps.

Lesson: Shrink the matrix with SVD

GloVe model equation

Exam favourite

$$w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$$

Each word has a word vector w, a context vector w̃ and two biases.

Lesson: GloVe: meaning lives in probability ratios

Ratio in vector form

$$(w_i - w_j) \cdot \tilde{w}_k = \log \frac{P(k \mid i)}{P(k \mid j)}$$

Differences of word vectors encode ratios, which is why analogies such as king − man + woman work.

Lesson: GloVe: meaning lives in probability ratios

GloVe objective

Exam favourite

$$J = \sum_{i,j} f(X_{ij}) \, (w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

Only pairs with $X_{ij} > 0$ enter, because $f(0) = 0$ and log 0 is undefined.

Lesson: The GloVe loss and its weighting function

Weighting function

Exam favourite

$$f(x) = (x \div x_{max})^{\alpha} \text{ if } x < x_{max}, \text{ else } 1$$

The paper uses $x_{max} = 100$ and $\alpha = 3/4$. The class notebook uses $x_{max} = 10$.

Lesson: The GloVe loss and its weighting function

Gradient for one pair

$$\frac{\partial J}{\partial w_i} = 2 \, f(X_{ij}) \, e_{ij} \, \tilde{w}_j, \quad \frac{\partial J}{\partial b_i} = 2 \, f(X_{ij}) \, e_{ij}$$

e_{ij} = prediction − $\log X_{ij}$, the error of that pair.

Lesson: The GloVe loss and its weighting function

Final vector

$$W_i = w_i + \tilde{w}_i$$

The two vectors play symmetric roles, so adding them (or averaging) is a little better than either alone.

Lesson: The GloVe loss and its weighting function

TABLES

Table: · i · like · enjoy

	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Lesson: Count the neighbours directly

Table: fix · what it does

Fix	What it does
log of the counts	squeezes 1000 and 10 into 6.9 and 2.3
min(X, t) with t ≈ 100	caps very frequent pairs
ignore function words	drops the, he, has from the counts
ramped windows	closer words count more than far ones
Pearson correlation, negatives set to 0	measures association instead of raw frequency

Lesson: Shrink the matrix with SVD

Table: probe word k · p(k | ice) · p(k | steam) · ratio

Probe word k	P(k | ice)	P(k | steam)	Ratio
solid	1.9×10^{-4}	2.2×10^{-5}	8.9
gas	6.6×10^{-5}	7.8×10^{-4}	8.5×10^{-2}
water	3.0×10^{-3}	2.2×10^{-3}	1.36
fashion	1.7×10^{-5}	1.8×10^{-5}	0.96

Lesson: GloVe: meaning lives in probability ratios

Table: step on the slide · what happens

Step on the slide	What happens
1 to 3	collect a corpus, choose a window (for example ±5), count X
4	take log X because raw counts are noisy and biased by frequency
5 and 6	predict $w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j$ and match it to log X_{ij}
7	weight each pair with $f(X_{ij})$
8	minimise the weighted squared error with gradient descent
9	use w_i or $(w_i + \tilde{w}_i) \div 2$ as the final vector

Lesson: The GloVe loss and its weighting function

Table: method · uses · strength · weakness

Method	Uses	Strength	Weakness
Count + SVD (LSA, COALS)	global counts	fast training, uses all statistics	poor at analogies, frequent words dominate
Prediction (word2vec)	local windows	good analogies, scales with the corpus	ignores global statistics, many passes
GloVe	global counts, trained with gradient descent	fast, good on small and large data	a big X must be stored

Lesson: The GloVe notebook and what it teaches

TIPS AND TRAPS

COMMON MISTAKE SVD costs about $O(V^3)$ time for a $V \times V$ matrix and must be redone when new words arrive. That is one reason prediction based methods such as word2vec took over, and GloVe tries to combine the two.

Lesson: Shrink the matrix with SVD

IN THE EXAM Exams like the table: given two probabilities, compute the ratio and say whether the probe word is an ice word, a steam word or neutral. 8.9 is an ice word, 0.085 a steam word, 1.36 and 0.96 neutral.

Lesson: GloVe: meaning lives in probability ratios

COMMON MISTAKE The loss is a weighted squared error on log counts, not a cross entropy. Students often write −log P like word2vec.

Lesson: The GloVe loss and its weighting function

REMEMBER The prediction inspection shows log 0.5 = −0.6931 for the pair (ice, water). Counts below 1 give negative targets. That is fine, because the target is a log count, not a probability.

Lesson: The GloVe notebook and what it teaches

2.5 Evaluating and improving embeddings

Intrinsic and extrinsic tests, fastText n-grams, cross-lingual mapping, sparse embeddings and debiasing.

KEY IDEAS

- **Intrinsic and extrinsic** Intrinsic: fast subtasks (relatedness with Spearman, analogy, categorisation with purity, selectional preference). Extrinsic: score of a real task.
- **Using pre-trained vectors** Fixed (fewer parameters, no adaptation), fine-tuned (adapts, may forget), concatenated (both, bigger). No embedding wins on every test.
- **Subwords** fastText: v_w = sum of character n-gram vectors with marks < >, n = 3 to 6. "where" has 14 n-grams. Handles rare and unseen words.
- **Cross-lingual** Map spaces with W, often orthogonal (Procrustes $W = UV^T$). Anchors: a dictionary, identical strings (Artetxe) or none (adversarial, Zhang).
- **Sparsity and bias** Sparse: add $\lambda\|w\|_1$. Debias: bias direction g from pairs, neutralise $v' = v - (v \cdot g)g$, then equalise. Bias is reduced, not removed.

FORMULAS

Spearman correlation (no ties)

Exam favourite

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

d_i is the difference between the human rank and the model rank of pair i.

Lesson: Intrinsic evaluation: test the vectors on their own

Analogy

Exam favourite

$$d = \text{argmax}_{w \notin \{a, b, c\}} \cos(w, b - a + c)$$

man : king :: woman : queen. The three input words are excluded.

Lesson: Intrinsic evaluation: test the vectors on their own

Cluster purity

$$\text{purity} = \frac{1}{N} \sum_{\text{clusters}} \max_{\text{class}} |\text{cluster} \cap \text{class}|$$

Each cluster votes for its majority class.

Lesson: Intrinsic evaluation: test the vectors on their own

Concatenation

$$x_w = [e_{pre}(w), e_{task}(w)] \in \mathbb{R}^{d_1 + d_2}$$

A 300-dimensional GloVe vector plus a learned 100-dimensional vector gives 400 inputs.

Lesson: Extrinsic evaluation: use the vectors in a real system

Trainable embedding parameters

$$\text{frozen: } 0, \quad \text{fine-tuned: } V \times d$$

With V = 10,000 and d = 300, fine-tuning trains 3 million extra numbers.

Lesson: Extrinsic evaluation: use the vectors in a real system

fastText word vector

Exam favourite

$$v_w = z_w + \sum_{g \in G(w)} z_g$$

G(w) is the set of character n-grams of w (usually n = 3 to 6) with boundary marks < and >.

Lesson: Subword embeddings and fastText

Number of n-grams

$$\sum_{n = lo}^{hi} (L + 2 - n + 1)$$

L is the word length. The marks add 2 characters.

Lesson: Subword embeddings and fastText

Training

$$\text{skip-gram score } s(w, c) = \sum_{g \in G(w)} z_g \cdot u_c$$

Same skip-gram or CBOW as word2vec, but the update goes to every n-gram vector of the word.

Lesson: Subword embeddings and fastText

Linear mapping

Exam favourite

$$\min_W \sum_i \|W x_i - y_i\|^2$$

x_i is a source word, y_i its translation.

Lesson: Knowledge, languages and alignment

Orthogonal Procrustes

$$W = U V^T \text{ where } U \Sigma V^T = \text{SVD}(Y X^T)$$

Forcing $W^T W = I$ (a rotation) keeps distances, and this closed form solves it.

Lesson: Knowledge, languages and alignment

Sparsity penalty

Exam favourite

$$J_{\text{sparse}} = J + \lambda \sum_i \|w_i\|_1$$

$\|w\|_1 = \sum |w_k|$. A bigger λ gives more zeros.

Lesson: Interpretable and fairer embeddings

Neutralise

Exam favourite

$$v' = v - (v \cdot g)\, g$$

g is a unit vector. After this v'·g = 0.

Lesson: Interpretable and fairer embeddings

TABLES

Table: · intrinsic · extrinsic

	Intrinsic	Extrinsic
Evaluated on	a specific intermediate subtask	a real task such as NER or translation
Speed	fast to compute	can take a long time
Strength	helps to understand the embedding	tells you if it really helps
Weakness	not clear it helps unless it correlates with a real task	unclear whether the embedding or another part is the problem

Lesson: Intrinsic evaluation: test the vectors on their own

Table: family · method · data and metric

Family	Method	Data and metric
Relatedness	cosine of word pairs against human similarity scores	WordSim-353, SimLex-999, Spearman or Pearson correlation
Analogy	b − a + c, nearest word by cosine	Google analogy test set, BATS, accuracy
Categorisation	cluster the vectors with k-means	purity or F-score against gold categories
Selectional preference	score verb and noun pairs	eat-apple should beat eat-rock

Lesson: Intrinsic evaluation: test the vectors on their own

Table: way to use pre-trained vectors · pros · cons

Way to use pre-trained vectors	Pros	Cons
Fixed (frozen)	fewer parameters, less overfitting	cannot adapt to task specific meaning
Fine-tuned	adapts to the task	may forget general knowledge
Concatenated [pre-trained, learned]	general and task specific information together	bigger model, more computation

Lesson: Extrinsic evaluation: use the vectors in a real system

Table: pieces · example · combined with

Pieces	Example	Combined with
characters	c, a, t, s	a BiLSTM or CNN over the characters
morphemes	un + fortunate + ly	an RNN or a sum
character n-grams (fastText)	<wh, whe, her, ere, re>, ...	a sum or average

Lesson: Subword embeddings and fastText

Table: method · anchor · how the mapping is found

Method	Anchor	How the mapping is found
Faruqui and Dyer 2014	a bilingual dictionary	project both spaces into a shared space (CCA)
Artetxe et al. 2017	identical strings: numbers, names, "COVID", "2025", "UN"	learn a linear map W from the anchors, then refine with mutual nearest neighbours
Zhang et al. 2017	none	adversarial training: a generator W against a discriminator that guesses the language

Lesson: Knowledge, languages and alignment

Table: step · what it does

Step	What it does
1. Identify	pick definitional pairs such as he and she, man and woman
2. Direction	compute the bias direction g from their differences (the top principal component)
3. Neutralise	remove the g part of words that should be neutral, such as programmer or nurse
(Equalise)	make each pair equally far from every neutral word

Lesson: Interpretable and fairer embeddings

TIPS AND TRAPS

IN THE EXAM A Spearman question gives two lists of scores. Rank both lists first, then use the formula. Pearson uses the raw scores instead.

Lesson: Intrinsic evaluation: test the vectors on their own

REMEMBER Pre-trained vectors help most when the labelled data is small. With a huge labelled set, vectors trained on the task can catch up.

Lesson: Extrinsic evaluation: use the vectors in a real system

COMMON MISTAKE The slide lists the trigrams of "playing" as <pl, pla, lay, ayi, yin, ing, ng>: 7 trigrams. The last one is ng>, the suffix shared by every -ing word.

Lesson: Subword embeddings and fastText

REMEMBER The adversarial method stops when the discriminator cannot tell a mapped English vector from a real Spanish one. At that point the two spaces are statistically indistinguishable.

Lesson: Knowledge, languages and alignment

COMMON MISTAKE Perfect debiasing remains hard. The slide says embeddings become more balanced, not unbiased. A balanced or filtered corpus is the other approach.

Lesson: Interpretable and fairer embeddings

STEP 3 NLP · Language models and RNNs

3.1 Language models and recurrent networks

n-gram LMs, the fixed window model, RNN equations and parameters, cross-entropy training and perplexity.

KEY IDEAS

- **Language model** Predicts P(next word | previous words). Chain rule gives the probability of a text. n-gram models keep only n − 1 words and count.
- **n-gram problems** Sparsity (fix: smoothing, backoff), storage, and a context that cannot exceed about 5 words. 4-gram example: 400 ÷ 1000 = 0.4.
- **RNN** $h^{(t)} = \tanh(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$, $\hat{y} = \text{softmax}(Uh + b_2)$. Same weights at each step. One-hot 5, hidden 6: 96 parameters.
- **Training** $J^{(t)} = -\log \hat{y}$ of the true next word, averaged over steps. Teacher forcing feeds gold words. −ln 0.6 = 0.51, −ln 0.05 = 2.99.
- **Perplexity** PP = exp(J), lower is better. PP = k means choosing among about k words: 1.44 confident, 4.07 confused.

FORMULAS

Next word distribution

Exam favourite

$$P(x^{(t+1)} \mid x^{(t)}, \ldots, x^{(1)})$$

$x^{(t+1)}$ can be any word of the vocabulary V.

Lesson: Language models and n-grams

Probability of a text (chain rule)

$$P(x^{(1)}, \ldots, x^{(T)}) = \prod_{t=1}^{T} P(x^{(t)} \mid x^{(t-1)}, \ldots, x^{(1)})$$

Exact. Every model below approximates the conditional terms.

Lesson: Language models and n-grams

n-gram (Markov) assumption

Exam favourite

$$P(x^{(t+1)} \mid x^{(t)}, \ldots, x^{(1)}) \approx P(x^{(t+1)} \mid x^{(t)}, \ldots, x^{(t-n+2)}) \approx \frac{\text{count}(x^{(t-n+2)} \ldots x^{(t+1)})}{\text{count}(x^{(t-n+2)} \ldots x^{(t)})}$$

Only the last n − 1 words matter. Probabilities come from counting.

Lesson: Language models and n-grams

Add-one (Laplace) smoothing

$$P(w \mid c) = \frac{\text{count}(c, w) + 1}{\text{count}(c) + |V|}$$

No probability is ever 0. With δ instead of 1 it is add-δ smoothing.

Lesson: Language models and n-grams

Fixed window model

Exam favourite

$$e = [e^{(1)}, e^{(2)}, e^{(3)}, e^{(4)}], \quad h = f(We + b_1), \quad \hat{y} = \text{softmax}(Uh + b_2)$$

$e^{(i)}$ is the embedding of the i-th window word. $\hat{y}$ is a distribution over V.

Lesson: A fixed window neural language model

Output matrix shape

$$U \in \mathbb{R}^{|V| \times d_h}$$

One row per vocabulary word. Row w scores how well the prefix fits word w.

Lesson: A fixed window neural language model

RNN hidden state

Exam favourite

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

$h^{(0)}$ is the initial state. σ is usually tanh. W_h and W_e are shared across time.

Lesson: Recurrent neural networks

RNN output

Exam favourite

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

The distribution over the next word after step t.

Lesson: Recurrent neural networks

Parameters (no biases, one-hot input)

$$|V| d_h + d_h^2 + d_h |V|$$

Input to hidden, hidden to hidden, hidden to output.

Lesson: Recurrent neural networks

Loss at step t

Exam favourite

$$J^{(t)}(\theta) = -\sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = -\log \hat{y}^{(t)}_{x^{(t+1)}}$$

Only the true word counts.

Lesson: Training: cross-entropy and teacher forcing

Overall loss

Exam favourite

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta)$$

An average over all steps, in practice over a batch of sentences.

Lesson: Training: cross-entropy and teacher forcing

Backpropagation through time

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial W_h}\Big|_{(i)}$$

W_h is used at every step, so its gradient is the sum of the gradients at each use.

Lesson: Training: cross-entropy and teacher forcing

Perplexity Exam favourite

$$PP = \prod_{t=1}^{T} \left(\frac{1}{P_{LM}(x^{(t+1)} \mid x^{(t)}, \ldots, x^{(1)})} \right)^{1/T} = \exp(J(\theta))$$

J is the average cross-entropy in natural logs.

Lesson: Perplexity and generating text

Meaning

$$PP = k \Leftrightarrow \text{the model is as unsure as a uniform choice among k words}$$

A perfect model has PP = 1. Guessing uniformly over V gives PP = |V|.

Lesson: Perplexity and generating text

TABLES

Table: problem · what goes wrong · partial fix

Problem	What goes wrong	Partial fix
Sparsity 1	the full n-gram never occurred, so a valid word gets probability 0	smoothing: add a small δ to every count
Sparsity 2	the context never occurred, so nothing can be computed	backoff: condition on fewer words, "opened their"
Storage	every seen n-gram must be stored	none: bigger n or corpus means a bigger model
Short context	n above 5 makes sparsity much worse	move to neural models

Lesson: Language models and n-grams

Table: improvement over n-grams · remaining problem

Improvement over n-grams	Remaining problem
no sparsity problem: every word gets a probability	the fixed window is too small
no need to store all observed n-grams	enlarging the window enlarges W
similar words share similar embeddings	$x^{(1)}$ and $x^{(2)}$ are multiplied by different parts of W, so there is no symmetry

Lesson: A fixed window neural language model

Table: rnn advantages · rnn disadvantages

RNN advantages	RNN disadvantages
any input length	recurrent computation is slow, steps cannot run in parallel
step t can in theory use information from many steps back	in practice it is hard to use information from many steps back
model size does not grow with the context	
the same weights at every step, so inputs are treated symmetrically	

Lesson: Recurrent neural networks

Table: perplexity · interpretation from the slide

Perplexity	Interpretation from the slide
1.44	on average the model chooses between about 1.44 words: confident and mostly right
4.07	the model is confused, almost guessing among 4 words

Lesson: Perplexity and generating text

TIPS AND TRAPS

IN THE EXAM Questions often ask for an n-gram probability from counts and then what happens when a count is 0. Say "sparsity", name smoothing and backoff, and give the Markov assumption.

Lesson: Language models and n-grams

COMMON MISTAKE The RNN parameter count never depends on the sentence length T. A common wrong answer multiplies by T.

Lesson: Recurrent neural networks

REMEMBER Why cross-entropy: logs turn products of probabilities into sums, confident wrong answers are penalised hard, and the gradient with softmax is simply $\hat{y} - y$, which is stable.

Lesson: Training: cross-entropy and teacher forcing

3.2 Sequence labelling, NER and bidirectional RNNs

RNN taggers, NER, BIO tags, span F1, the parameter questions and bidirectional RNNs.

KEY IDEAS

- **Sequence labelling** One softmax over K tags per word, trained with cross-entropy. POS tagging is the classic case. RNN uses: generate, label, classify.
- **NER** Find spans that are proper names and their type: PER, LOC, ORG, GPE (also dates, prices). Hard because of segmentation and type ambiguity.
- **BIO** B starts a span, I continues it, O is outside. 2n + 1 tags (IO n + 1, BIOES 4n + 1). Score NER with exact span F1.
- **Parameter answers** E·H + H·H + H·K: 300, 50, 50 tags gives 20,000. NER with 5 types: 11 labels, 18,050. BiRNN doubles the RNN part and reads 2H.
- **BiRNN** $h^{(t)} = [h\rightarrow^{(t)}, h\leftarrow^{(t)}]$: each word sees the whole sentence. Not for language modelling. Sentence vector: final, mean or max state.

FORMULAS

Tag distribution at step t — Exam favourite

$$\hat{y}^{(t)} = \text{softmax}(V\, h^{(t)}) \in \mathbb{R}^K$$

K is the number of tags. $h^{(t)}$ is the RNN state after reading word t.

Lesson: Sequence labelling with an RNN

Training loss

$$J = -\frac{1}{T} \sum_t \log \hat{y}^{(t)}_{y_t}$$

Cross-entropy with the gold tag at every position.

Lesson: Sequence labelling with an RNN

Prediction

$$\text{tag}_t = \text{argmax}_k\, \hat{y}_k^{(t)}$$

One independent choice per word.

Lesson: Sequence labelling with an RNN

Number of BIO tags — Exam favourite

$$2n + 1$$

n B tags, n I tags and 1 O tag, where n is the number of entity types.

Lesson: BIO tagging: spans become one tag per word

Other schemes

$$\text{IO: } n + 1, \quad \text{BIOES: } 4n + 1$$

BIOES adds E for the last word and S for a single word span.

Lesson: BIO tagging: spans become one tag per word

Span precision and recall — Exam favourite

$$P = \frac{\text{correct spans}}{\text{predicted spans}}, \quad R = \frac{\text{correct spans}}{\text{gold spans}}$$

F1 = 2PR ÷ (P + R).

Lesson: BIO tagging: spans become one tag per word

RNN tagger parameters (no biases)

Exam favourite

$$E \cdot H + H \cdot H + H \cdot K$$

E embedding size, H hidden size, K number of output labels. The embedding table itself is not counted here.

Lesson: The two parameter questions

BiRNN tagger

$$2(E \cdot H + H \cdot H) + 2H \cdot K$$

Two directions, and the output layer reads the joined 2H vector.

Lesson: The two parameter questions

Forward RNN

$$h_{\rightarrow}^{(t)} = \mathrm{RNN}_{\mathrm{FW}}(h_{\rightarrow}^{(t-1)}, x^{(t)})$$

Reads from the start.

Lesson: Sentence classification and bidirectional RNNs

Backward RNN

$$h_{\leftarrow}^{(t)} = \mathrm{RNN}_{\mathrm{BW}}(h_{\leftarrow}^{(t+1)}, x^{(t)})$$

Reads from the end, with its own weights.

Lesson: Sentence classification and bidirectional RNNs

Joined state

Exam favourite

$$h^{(t)} = [h_{\rightarrow}^{(t)}, h_{\leftarrow}^{(t)}] \in \mathbb{R}^{2H}$$

Each word now sees the whole sentence.

Lesson: Sentence classification and bidirectional RNNs

Sentence vector

$$s = \frac{1}{T} \sum_t h^{(t)} \quad \text{or} \quad s = h^{(T)}$$

Then $\hat{y}$ = softmax(W s).

Lesson: Sentence classification and bidirectional RNNs

TABLES

Table: use of an rnn · output · example

Use of an RNN	Output	Example
Generation	next word at every step	language model
Sequence labelling	one label per word	POS tagging, NER
Classification	one label per sentence	sentiment, spam

Lesson: Sequence labelling with an RNN

Table: tag · type · example

Tag	Type	Example
PER	person	Marie Curie
LOC	location	New York City
ORG	organisation	Stanford University
GPE	geo-political entity	Boulder, Colorado

Lesson: Named entity recognition

Table: why ner is hard · example

Why NER is hard	Example
Segmentation: POS gives one tag per word, NER must find where each entity starts and ends	"United Airlines Holding" is one entity of three words
Type ambiguity: the same string can be different types	Washington: a person, a city (GPE), a state, or the government (ORG)

Lesson: Named entity recognition

Table: word · jane · villanueva · of

Word	Jane	Villanueva	of	United	,	United	Airlines	Holding	Chicago
BIO	B-PER	I-PER	O	B-ORG	O	B-ORG	I-ORG	I-ORG	B-LOC

Lesson: BIO tagging: spans become one tag per word

Table: · unidirectional · bidirectional

	Unidirectional	Bidirectional
Sees at word t	words 1 to t	the whole sentence
Good for	language modelling and generation	tagging, NER, classification, encoders
Not usable for		language modelling, because the future is not known yet

Lesson: Sentence classification and bidirectional RNNs

TIPS AND TRAPS

REMEMBER — A left to right RNN tagger only sees the past when it tags word t. "back" in "the back of the bill" needs the words after it, which is what bidirectional RNNs add.

Lesson: Sequence labelling with an RNN

COMMON MISTAKE — Why B is needed: with IO tags "[ORG Google] [ORG Microsoft]" becomes I-ORG I-ORG and looks like one entity. B marks the start of the second one.

Lesson: BIO tagging: spans become one tag per word

IN THE EXAM — State what you count. If the question says "excluding bias terms" and does not mention the embedding table, count only the RNN and output weights, as the lecture answer does.

Lesson: The two parameter questions

3.3 Vanishing gradients, LSTM and GRU

Vanishing gradients, LSTM and GRU equations, why gating works, parameter counts and xLSTM.

KEY IDEAS

- **Vanishing and exploding** BPTT multiplies one Jacobian $\text{diag}(\sigma')W_h$ per step: below 1 it vanishes, above 1 it explodes. Fixes: ReLU, initialisation, residual links, LSTM/GRU, clipping for exploding.
- **LSTM equations** f, i, o = σ(Wh + Ux + b), c̃ = tanh(...), $c = f \odot c_{old} + i \odot \tilde{c}$, $h = o \odot \tanh(c)$. Parameters 4H(E + H) + 4H.
- **Why LSTM works** Additive cell path: $\partial c^{(t)}/\partial c^{(t-1)} \approx f$. With f near 1 information and gradients survive many steps.
- **GRU** z and r gates, $\tilde{h} = \tanh(Wx + U(r \odot h) + b)$, $h = (1 - z) \odot h + z \odot \tilde{h}$. Parameters 3H(E + H). Faster, fewer parameters.
- **BiLSTM answer** E = 60, H = 32 each way, 4 tags: 2 × 4 × 32 × 92 + 64 × 4 = 23,808. RNNs cannot parallelise over time, so attention and transformers followed.

FORMULAS

One step back — Exam favourite

$$\frac{\partial h^{(t)}}{\partial h^{(t-1)}} = \text{diag}(\sigma'(W_h h^{(t-1)} + W_x x^{(t)} + b))\, W_h$$

The Jacobian of one RNN step.

Lesson: Vanishing and exploding gradients

Many steps back — Exam favourite

$$\frac{\partial J^{(t)}}{\partial h^{(j)}} = \frac{\partial J^{(t)}}{\partial h^{(t)}} \prod_{j < k \le t} \frac{\partial h^{(k)}}{\partial h^{(k-1)}}$$

A product of t − j Jacobians.

Lesson: Vanishing and exploding gradients

Size bound

$$\|\partial h^{(t)} \div \partial h^{(j)}\| \leq (\gamma \, \|W_h\|)^{t-j}$$

γ bounds σ' (1 for tanh). Below 1 the bound shrinks exponentially.

Lesson: Vanishing and exploding gradients

Gradient clipping (for exploding gradients)

$$\text{if } \|g\| > \tau\text{:} \quad g \leftarrow \frac{\tau}{\|g\|} \, g$$

Keeps the direction, limits the step size.

Lesson: Vanishing and exploding gradients

Forget gate

Exam favourite

$$f^{(t)} = \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f)$$

What to keep from the old cell.

Lesson: Long Short-Term Memory

Input gate

$$i^{(t)} = \sigma(W_i h^{(t-1)} + U_i x^{(t)} + b_i)$$

How much of the new content to write.

Lesson: Long Short-Term Memory

Output gate

$$o^{(t)} = \sigma(W_o h^{(t-1)} + U_o x^{(t)} + b_o)$$

How much of the cell to show as h.

Lesson: Long Short-Term Memory

New cell content

$$\tilde{c}^{(t)} = \tanh(W_c h^{(t-1)} + U_c x^{(t)} + b_c)$$

The candidate to write.

Lesson: Long Short-Term Memory

Cell state

Exam favourite

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

Erase some, write some. ⊙ is element-wise.

Lesson: Long Short-Term Memory

Hidden state

Exam favourite

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

What the next layer and the next step see.

Lesson: Long Short-Term Memory

LSTM layer parameters

Exam favourite

$$4 \cdot H \cdot (E + H) + 4H$$

Four blocks (f, i, o, c̃), each with an H × E input matrix, an H × H recurrent matrix and a bias.

Lesson: Long Short-Term Memory

Direct cell path

Exam favourite

$$\frac{\partial c^{(t)}}{\partial c^{(t-1)}} \approx \text{diag}(f^{(t)})$$

Ignoring the paths through the gates. With f = 1 the cell is a straight wire.

Lesson: How the LSTM keeps gradients alive

Over many steps

$$\prod_k f^{(k)}$$

The network learns f. Near 1 it keeps information for many steps, near 0 it forgets on purpose.

Lesson: How the LSTM keeps gradients alive

Update gate

Exam favourite

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

How much new information to store.

Lesson: Gated Recurrent Units

Reset gate

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

How much of the past to forget when building the candidate.

Lesson: Gated Recurrent Units

Candidate

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

New information from the input and the filtered past.

Lesson: Gated Recurrent Units

New state Exam favourite

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

A blend of old and new. z = 0 keeps the old state exactly.

Lesson: Gated Recurrent Units

TABLES

Table: consequence (slide 72) · fix

Consequence (slide 72)	Fix
early layers and early time steps learn very slowly	ReLU activations
deep networks become hard to train	suitable weight initialisation, for example orthogonal W_h
long dependencies are lost	residual or more direct connections, attention
memory fades in a plain RNN	LSTM and GRU for sequential data

Lesson: Vanishing and exploding gradients

Table: · rnn · lstm

	RNN	LSTM
memory	$h^{(t)}$, rewritten every step	separate cell $c^{(t)}$, changed only through gates
gradient through one step	W_h times tanh'	f, learned per step
to keep a fact	weights must be tuned just right, which is very hard	set the forget gate to 1 and the input gate to 0

Lesson: How the LSTM keeps gradients alive

Table: · lstm · gru

	LSTM	GRU
gates	forget, input, output	update, reset
memory	cell c and state h	state h only
parameter blocks	4	3
parameters (E = 100, H = 64, no biases)	4 × 64 × 164 = 41,984	3 × 64 × 164 = 31,488
when to use	default, long dependencies, plenty of data	faster, fewer parameters, small data

Lesson: Gated Recurrent Units

Table: xlstm (beck et al. 2024) · what changes

xLSTM (Beck et al. 2024)	What changes
exponential gating	gates can grow beyond 1 with a normaliser, so the model can revise a stored decision more strongly
sLSTM	scalar memory with memory mixing between heads
mLSTM	a matrix memory updated with a covariance rule, fully parallelisable
result	LSTM style models that scale close to transformers on language modelling

Lesson: BiLSTM, the parameter question and xLSTM

TIPS AND TRAPS

COMMON MISTAKE Vanishing gradients mean the model does not learn long range dependencies. They do not mean the loss is 0 or that training stops at once. Exploding gradients show up as NaN losses.

Lesson: Vanishing and exploding gradients

IN THE EXAM An LSTM does not guarantee there is no vanishing or exploding gradient. It makes it much easier for the model to learn long range dependencies. In practice it handles about 100 steps well where a plain RNN manages about 7.

Lesson: How the LSTM keeps gradients alive

REMEMBER In a GRU, (1 – z) plays the role of the LSTM forget gate and z the role of the input gate. Tying them means writing new information always erases the same amount of old information.

Lesson: Gated Recurrent Units

STEP 4 NLP · Translation, attention and decoding

4.1 Sequence-to-sequence models and machine translation

Encoder and decoder, conditional LM, training, uses, the bottleneck, low-resource NMT and parameter counting.

KEY IDEAS

- **Seq2seq** Encoder RNN reads x, its last state starts the decoder RNN, which writes y one word at a time and feeds its outputs back in at test time.
- **Conditional LM** $P(y \mid x) = \prod P(y_t \mid y_{<t}, x)$. Language model: predicts the next target word. Conditional: on the whole source. Trained on a parallel corpus.
- **Uses** Translation, summarisation, dialogue, parsing, code generation: any task whose input and output are sequences.
- **Bottleneck** The whole source passes through one vector, so long sentences lose information. Attention fixes it.
- **Low-resource and parameters** Transfer, multilingual, back translation, augmentation, pivot, unsupervised. Spanish to French count with stated assumptions: 38,005,760, nearly all in embeddings and output.

FORMULAS

Encoder

$$h_i^{enc} = RNN_{enc}(h_{i-1}^{enc}, x_i), \quad i = 1 \dots n$$

Reads the source word by word.

Lesson: Neural machine translation

Decoder Exam favourite

$$s_t = RNN_{dec}(s_{t-1}, y_{t-1}), \quad s_0 = h_n^{enc}$$

Starts from the encoding and reads its own previous output.

Lesson: Neural machine translation

Output

$$P(y_t \mid y_{<t}, x) = \text{softmax}(W_o\, s_t)$$

A distribution over the target vocabulary.

Lesson: Neural machine translation

NMT probability Exam favourite

$$P(y \mid x) = \prod_{t=1}^{T} P(y_t \mid y_1, \dots, y_{t-1}, x)$$

Each factor conditions on the target words so far and on the entire source sentence.

Lesson: A conditional language model

Training loss Exam favourite

$$J = \frac{1}{T} \sum_{t=1}^{T} -\log P(y_t^{gold} \mid y_{<t}^{gold}, x)$$

Teacher forcing: the decoder reads the gold previous word.

Lesson: A conditional language model

TABLES

Table: part · what it does

Part	What it does
Encoder RNN	reads the source sentence "il a m' entarté" and produces an encoding of it
Encoding	the last encoder state, used as the initial hidden state of the decoder
Decoder RNN	a language model that writes the target "he hit me with a pie", conditioned on the encoding
At test time	each decoder output (the argmax word) is fed back in as the next input until <END>

Lesson: Neural machine translation

Table: task · input sequence · output sequence

Task	Input sequence	Output sequence
Summarisation	long text	short text
Dialogue	previous utterances	next utterance
Parsing	input text	the parse written as a sequence
Code generation	natural language	Python code
Speech recognition	audio frames	words
Transliteration	Hindi in Devanagari letters	the same words in Latin letters

Lesson: Seq2seq is versatile

Table: approach · idea

Approach	Idea
Transfer learning	train on a high-resource pair first, then fine-tune on the low-resource data
Multilingual NMT	one model for many languages, low-resource languages share representations
Back translation	translate monolingual target text back into the source to make synthetic parallel data
Data augmentation	paraphrasing, noise injection, word replacement
Pivot translation	A to English to B when direct A to B data is scarce
Unsupervised NMT	only monolingual data, no parallel corpus at all

Lesson: Low-resource NMT and the parameter question

Table: component · shape · count

Component	Shape	Count
Source embeddings	35,000 × 256	8,960,000
Target embeddings	45,000 × 256	11,520,000
Encoder RNN	256 × 128 + 128 × 128	49,152
Bridge, encoder to decoder state	128 × 256	32,768
Decoder RNN	256 × 256 + 256 × 256	131,072
Attention projection (sizes differ)	256 × 128	32,768
Output on $[s_t, c_t]$	384 × 45,000	17,280,000
Total		38,005,760

Lesson: Low-resource NMT and the parameter question

TIPS AND TRAPS

IN THE EXAM In the exam, name both parts: "language model" because it predicts the next target word, "conditional" because it conditions on the whole source x.

Lesson: A conditional language model

REMEMBER Encoder and decoder do not need to be RNNs. The encoder can be a CNN over an image (image captioning) and both can be transformers.

Lesson: Seq2seq is versatile

IN THE EXAM Most of the parameters sit in the embeddings and the output layer, about 37.8 million of 38 million. The RNNs themselves are tiny. Variants change the total: concatenating the context to the decoder input adds 128 × 256 = 32,768, and a decoder with the same size as the encoder needs no bridge or projection.

Lesson: Low-resource NMT and the parameter question

4.2 Attention in encoder-decoder models

Attention steps, the lecture problem, scoring variants and the benefits of attention.

KEY IDEAS

- **Attention idea** At each decoder step, look directly at all encoder states and focus on the relevant ones. A dynamic context per step solves the bottleneck.
- **Equations** $e_i = s_t^T h_i$, $\alpha = \text{softmax}(e)$, $a_t = \sum \alpha_i h_i$, then predict from $[a_t, s_t]$.
- **Lecture problem** Scores 2, −5, 3, −2, weights 0.2676, 0.0002, 0.7273, 0.0049, context ≈ [1.262, −0.722].
- **Variants** Dot (same size), multiplicative $s^T W h$, reduced rank $(Us)^T(Vh)$, additive $v^T \tanh(W_1 h + W_2 s)$, scaled by $1 \div \sqrt{d}$.
- **Benefits** Better NMT, no bottleneck, gradient shortcuts, interpretable weights and soft alignment learned for free. Leads to self-attention and transformers.

FORMULAS

1. Scores Exam favourite

$$e^t = [s_t^T h_1, \ldots, s_t^T h_N] \in \mathbb{R}^N$$

One dot product per source position.

Lesson: Attention in equations

2. Attention distribution Exam favourite

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

Non-negative weights that add to 1.

Lesson: Attention in equations

3. Attention output (context) Exam favourite

$$a_t = \sum_{i=1}^{N} \alpha_i^t h_i \in \mathbb{R}^d$$

A weighted average of the encoder states.

Lesson: Attention in equations

4. Use it

$$\hat{y}_t = \text{softmax}(W_o[a_t, s_t])$$

Concatenate the context with the decoder state and continue as before.

Lesson: Attention in equations

Basic dot product Exam favourite

$$e_i = s^T h_i$$

Needs $d_s = d_h$. No parameters.

Lesson: Ways to compute the scores

Multiplicative (bilinear, Luong) Exam favourite

$$e_i = s^T W h_i, \quad W \in \mathbb{R}^{d_s \times d_h}$$

Works for different sizes. $d_s \times d_h$ parameters.

Lesson: Ways to compute the scores

Reduced rank multiplicative

$$e_i = (Us)^T(Vh_i), \quad U \in \mathbb{R}^{k \times d_s}, V \in \mathbb{R}^{k \times d_h}$$

k much smaller than d. The form transformers use.

Lesson: Ways to compute the scores

Additive (Bahdanau) Exam favourite

$$e_i = v^T \tanh(W_1 h_i + W_2 s)$$

A small neural network scores each pair. d_a is its hidden size.

Lesson: Ways to compute the scores

Scaled dot product

$$e_i = \frac{s^T h_i}{\sqrt{d}}$$

Keeps scores from growing with d, so the softmax does not saturate.

Lesson: Ways to compute the scores

TABLES

Table: · without attention · with attention

	Without attention	With attention
what the decoder sees of the source	one vector, the last encoder state	a fresh weighted mix of all encoder states at every step
long sentences	quality drops	quality stays high
alignment	hidden	visible in the attention weights

Lesson: The idea of attention

Table: h i · score · weight · reading

h_i	score	weight	reading
[2, 0]	2	0.2676	second most relevant
[−1, 2]	−5	0.0002	points away from s, ignored
[1, −1]	3	0.7273	most aligned with s, dominates
[0, 1]	−2	0.0049	almost ignored

Lesson: The lecture problem, fully worked

Table: variant · parameters (d h = 128, d s = 256, d a = 64)

Variant	Parameters (d_h = 128, d_s = 256, d_a = 64)
dot product	0, and it cannot be used because the sizes differ
multiplicative	256 × 128 = 32,768
additive	64 × 128 + 64 × 256 + 64 = 24,640

Lesson: Ways to compute the scores

Table: benefit · why

Benefit	Why
better translations	the decoder can focus on the relevant source words
solves the bottleneck	the decoder reads the source directly instead of through one vector
helps vanishing gradients	there is a shortcut from every decoder step to every source word
interpretability	the attention weights show what the decoder was looking at
alignment for free	the model learns which source word matches which target word without being told

Lesson: Why attention is so helpful

TIPS AND TRAPS

COMMON MISTAKE The weights depend on the decoder state, so they change at every output step. That is what makes the context dynamic. A common exam slip is to compute one set of weights for the whole sentence.

Lesson: Attention in equations

IN THE EXAM Show the scores, the softmax and the weighted sum as three separate lines. Keep 4 decimals in the weights, and check that they add to 1 (0.2676 + 0.0002 + 0.7273 + 0.0049 = 1.0000).

Lesson: The lecture problem, fully worked

4.3 Decoding: greedy, beam search and sampling

Greedy and beam search, stopping and length normalisation, and temperature, top-k and top-p sampling.

KEY IDEAS

- **Greedy** Pick argmax at each step. Fast, cannot undo decisions. Slide tree: greedy "a cat" 0.20, optimal "the cat" 0.36.
- **Beam search** Keep the k best hypotheses by ∑ log P, expand each, keep the top k again. k = 1 is greedy. Not guaranteed optimal, far cheaper than $|V|^T$.
- **Stopping and length** A hypothesis that emits <END> is set aside. Stop at time T or after n complete hypotheses. Pick the best by (1 ÷ t) ∑ log P.
- **Sampling** Temperature softmax(s ÷ T): low T sharp, high T flat. Top-k keeps k words, top-p the smallest set with mass ≥ p. Adds diversity.
- **Trade off** Greedy and beam are deterministic and can repeat in open ended text. Sampling trades some quality for diversity.

FORMULAS

Greedy step Exam favourite

$$\hat{y}_t = \text{argmax}_{w \in V} P(w \mid \hat{y}_{<t}, x)$$

One choice per step, never revisited.

Lesson: Greedy decoding and its problem

What we really want Exam favourite

$$\hat{y} = \text{argmax}_y P(y \mid x) = \text{argmax}_y \prod_t P(y_t \mid y_{<t}, x)$$

The best whole sequence, not the best word at each step.

Lesson: Greedy decoding and its problem

Hypothesis score Exam favourite

$$\text{score}(y_1, \ldots, y_t) = \log P(y_{1:t} \mid x) = \sum_{i=1}^{t} \log P(y_i \mid y_{<i}, x)$$

All scores are negative. Higher is better. Logs turn products into sums.

Lesson: Beam search

One beam step Exam favourite

expand each of the k hypotheses by every word, keep the top k of the $k \cdot |V|$ candidates

k = 1 is greedy decoding.

Lesson: Beam search

Length normalised score Exam favourite

$$\frac{1}{t} \sum_{i=1}^{t} \log P(y_i \mid y_{<i}, x)$$

Average log probability per word, used to pick the final answer.

Lesson: Stopping, length and cost

Cost per step

exhaustive: $|V|^T$ sequences in total, beam: $k \cdot |V|$ candidates per step

Beam search is linear in T.

Lesson: Stopping, length and cost

Pure sampling

$$y_t \sim P(\cdot \mid y_{<t}, x)$$

Draw the next word at random from the full distribution.

Lesson: Sampling: temperature, top-k and top-p

Temperature Exam favourite

$$P_T(w) = \frac{\exp(s_w \div T)}{\sum_{w'} \exp(s_{w'} \div T)}$$

T < 1 sharpens (more greedy), T > 1 flattens (more diverse). T → 0 is greedy.

Lesson: Sampling: temperature, top-k and top-p

Top-k Exam favourite

sample only from the k most probable words, renormalised

A fixed number of candidates.

Lesson: Sampling: temperature, top-k and top-p

Top-p (nucleus) Exam favourite

sample from the smallest set V_p with $\sum_{w \in V_p} P(w) \geq p$

The number of candidates adapts to how sure the model is.

Lesson: Sampling: temperature, top-k and top-p

TABLES

Table: stopping criterion (slide 117) · meaning

Stopping criterion (slide 117)	Meaning
reach a time step T	a preset maximum length
have n completed hypotheses	a preset number of finished candidates

Lesson: Stopping, length and cost

Table: decoder · deterministic · quality · diversity

Decoder	Deterministic	Quality	Diversity	Typical use
greedy	yes	good for short outputs	none	quick baselines
beam search	yes	high for translation and summaries	low, can repeat	machine translation
pure sampling	no	can drift into nonsense	high	rarely used alone
temperature, top-k, top-p	no	good balance	tunable	chatbots, story writing

Lesson: Sampling: temperature, top-k and top-p

TIPS AND TRAPS

IN THE EXAM Greedy is fast and often fine, but it is not optimal. Exams ask for an example where greedy fails: show a first choice with a high probability followed by low probabilities, against a lower first choice followed by a high one.

Lesson: Greedy decoding and its problem

COMMON MISTAKE Beam search is not guaranteed to find the optimal solution. It is much more efficient than exhaustive search, and a larger k gets closer to the optimum at a higher cost.

Lesson: Beam search

REMEMBER This is the quality against diversity trade off. Lower T, smaller k or smaller p give safer and more repetitive text. Higher values give more varied and riskier text.

Lesson: Sampling: temperature, top-k and top-p